

Template for XCPC

Dft Fan Club

2025 ∼ 2026	Verifying We Are Human.	Chaoyi Sun, Liwen Zhang, Tian Zhou
2026 ∼ 2027	Human verification passed.	Chaoyi Sun, Liwen Zhang, Fengting Deng

Contents

1 Graph . . . **1**
- 1.1 Diameter of a Tree . . . 1
- 1.2 Centroid of a Tree . . . 1
- 1.3 Minimum Spanning Tree . . . 1
  - 1.3.1 Prim . . . 1
  - 1.3.2 Kruskal . . . 1
- 1.4 LCA . . . 1
- 1.5 Virtual Tree . . . 1
- 1.6 Topological Sorting . . . 2
- 1.7 Shortest Path . . . 2
  - 1.7.1 Dijkstra . . . 2
  - 1.7.2 SPFA . . . 2
- 1.8 DCC . . . 3
- 1.9 Edge-Biconnected Components . . . 3
- 1.10 Tarjan . . . 3
- 1.11 Bipartite Graph Matchings . . . 4
- 1.12 Flow . . . 4
  - 1.12.1 Edmonds-Karp . . . 4
  - 1.12.2 Dinic . . . 4

2 Data Structure . . . **6**
- 2.1 Segment Tree . . . 6
- 2.2 Fenwick Tree . . . 6
- 2.3 Bitset . . . 6
- 2.4 Heavy-Light Decomposition . . . 7
- 2.5 Longest Chain Decomposition . . . 7
- 2.6 Link Cut Tree . . . 7
- 2.7 Treap . . . 8
- 2.8 Cartesian Tree . . . 9
- 2.9 Sparse Table . . . 9
- 2.10 Persistent Data Structure . . . 9
  - 2.10.1 Persistent Segment Tree . . . 9
  - 2.10.2 Persistent Treap . . . 9
- 2.11 Linear-Basis . . . 10

3 Math . . . **11**
- 3.1 Subset Enumeration . . . 11
- 3.2 Elementary Arithmetic Bucket . . . 11
- 3.3 Fraction . . . 11
- 3.4 Combination . . . 11
- 3.5 The Sieve of Primes & Euler's Totient Function & MÃűbius Function . . . 11
- 3.6 Exgcd . . . 11
- 3.7 CRT . . . 11
- 3.8 Lagrange Interpolation . . . 12
- 3.9 Max-Plus Convolution . . . 12
- 3.10 Gaussian Elimination . . . 12
- 3.11 FFT . . . 12
- 3.12 NTT . . . 12
- 3.13 FWT . . . 13

4 Computational Geometry . . . **14**

5 String . . . **16**
- 5.1 Hash . . . 16
- 5.2 KMP . . . 16
- 5.3 Manacher . . . 16
- 5.4 Trie . . . 16
  - 5.4.1 Trie . . . 16
  - 5.4.2 01-Trie . . . 16
- 5.5 Aho-Corasick Automaton . . . 16
- 5.6 SA . . . 17
- 5.7 SAM . . . 17

6 Other . . . **19**
- 6.1 Fast I/O . . . 19
- 6.2 Random Number Generator . . . 19

6.3 Execution 19
6.4 Comparison 19
6.5 Python 19

1 Graph

1.1 Diameter of a Tree

```cpp
template <typename T>
class Diameter
{
    int n, p;
    vector <T> dis;
    vector <vector <pair <int, T>>> ve;
    void dfs(int u, int fa)
    {
        for (const auto& [v, w] : ve[u])
        {
            if (v == fa) continue;
            dis[v] = dis[u] + w;
            if (dis[v] > dis[p]) p = v;
            dfs(v, u);
        }
    }
    public:
    Diameter(int n) : n(n), ve(n + 1) {}
    void add(int u, int v, T w = 1) {ve[u].push_back({v, w}); ve[v].
      push_back({u, w});}
    T calc()
    {
        dis.assign(n + 1, 0);
        p = 1; dfs(1, 0);
        dis[p] = 0; dfs(p, 0);
        return dis[p];
    }
};
```

1.2 Centroid of a Tree

```cpp
class Centroid
{
    int n;
    vector <int> sz, w, cen;
    vector <vector <int>> ve;
    void dfs(int u, int fa)
    {
        sz[u] = 1; w[u] = 0;
        for (auto v : ve[u])
        {
            if (v == fa) continue;
            dfs(v, u);
            sz[u] += sz[v];
            w[u] = max(w[u], sz[v]);
        }
        w[u] = max(w[u], n - sz[u]);
        if (w[u] <= n / 2) cen.push_back(u);
    }
    public:
    Centroid(int n) : n(n), ve(n + 1), sz(n + 1), w(n + 1) {}
    void add(int u, int v) {ve[u].push_back(v); ve[v].push_back(u);}
    auto calc()
    {
        cen.clear();
        dfs(1, 0);
        sort(cen.begin(), cen.end());
        return cen;
    }
};
```

1.3 Minimum Spanning Tree

1.3.1 Prim

```cpp
template <typename T>
class MST
{
    int n; T ans;
    vector <int> vis; vector <vector <int>> g; vector <T> dis;
    public :
    MST(int n) : n(n), vis(n + 1, 0), g(n + 1, vector <int> (n + 1, INF))
      , dis(n + 1, INF) {dis[1] = ans = 0; vis[1] = 1;}
    void add(int u, int v, T w) {g[u][v] = g[v][u] = w;}
    T calc()
    {
        for (int i = 2; i <= n; ++i) dis[i] = g[1][i];
        for (int i = 1; i < n; ++i)
        {
            int k = 0;
            for (int j = 1; j <= n; ++j)
                if (!vis[j] && dis[j] < dis[k]) k = j;
            vis[k] = 1;
            ans += dis[k];
            for (int j = 1; j <= n; ++j)
                if (!vis[j] && g[k][j] < dis[j]) dis[j] = g[k][j];
        }
        return ans;
    }
};
```

1.3.2 Kruskal

```cpp
template <typename T>
class MST
{
    int n, m, e_cnt, cnt; T ans;
    struct node {int u, v; T w;};
    vector <int> fa; vector <node> g;
    public:
    MST(int n, int m) : n(n), m(m), fa(n + 1, 0), g(m + 1) {cnt = e_cnt
      = ans = 0;}
    void add(int u, int v, int w) {g[++e_cnt].u = u, g[e_cnt].v = v, g[
      e_cnt].w = w;}
    int getfa(int u) {return fa[u] == u ? u : fa[u] = getfa(fa[u]);}
    T calc()
    {
        sort(g.begin(), g.end(), [] (auto &x, auto &y) {return x.w < y.w
      ;});
        for (int i = 1; i <= n; ++i) fa[i] = i;
        for (int i = 1; cnt < n && i <= m; ++i)
        {
            int dx = getfa(g[i].u), dy = getfa(g[i].v);
            if (dx == dy) continue;
            ans += g[i].w; fa[dx] = dy; ++cnt;
        }
        return ans;
    }
};
```

1.4 LCA

```cpp
class LCA
{
    static constexpr int lg = 20;
    int n;
    vector <int> dep;
    vector <vector <int>> f, ve;
    public:
    LCA(int n) : n(n), ve(n + 1), dep(n + 1), f(n + 1, vector <int> (lg +
       1, 0)) {}
    void add(int u, int v) {ve[u].push_back(v); ve[v].push_back(u);}
    void pre(int u, int fa)
    {
        f[u][0] = fa; dep[u] = dep[fa] + 1;
        for (int i = 0; i < lg; ++i) f[u][i + 1] = f[f[u][i]][i];
        for (auto v : ve[u])
            if (v != fa) pre(v, u);
    }
    int query(int u, int v)
    {
        if (dep[u] < dep[v]) swap(u, v);
        for (int i = lg; ~i; --i)
        {
            if (dep[f[u][i]] >= dep[v]) u = f[u][i];
            if (u == v) return u;
        }
        for (int i = lg; ~i; --i)
            if (f[u][i] != f[v][i]) u = f[u][i], v = f[v][i];
        return f[u][0];
    }
};
```

1.5 Virtual Tree

```cpp
typedef long long ll;
typedef pair <int, ll> pil;

class VT
{
    int n = 0, lg = 0;
    vector <int> dfn, out;
    vector <ll> dis;
    vector <vector <int>> f;
    bool anc(int u, int v) const
    {
        return dfn[u] <= dfn[v] && dfn[v] <= out[u];
    }
    int lca(int u, int v) const
    {
        if (anc(u, v)) return u;
        if (anc(v, u)) return v;
```

```
        for (int j = lg - 1; j >= 0; --j)
            if (!anc(f[j][u], v)) u = f[j][u];
        return f[0][u];
    }
    public:
    // G[1..n]: undirected weighted tree; G[0] unused.
    // Root at 1. O(n log n) time / space, no recursion.
    void init(const vector <vector <pil>> &G)
    {
        n = max(0, (int)G.size() - 1);
        lg = 1;
        while ((1LL << lg) <= n) ++lg;
        dfn.assign(n + 1, 0); out.assign(n + 1, 0);
        dis.assign(n + 1, 0);
        f.assign(lg, vector <int> (n + 1, 1));
        if (!n) return;
        int timer = 0;
        vector <int> st = {1};
        while (!st.empty())
        {
            int u = st.back(); st.pop_back();
            if (u < 0) {out[-u] = timer; continue;}
            dfn[u] = ++timer;
            for (int j = 1; j < lg; ++j)
                f[j][u] = f[j - 1][f[j - 1][u]];
            st.push_back(-u);
            for (auto [v, w] : G[u])
                if (v != f[0][u])
                {
                    f[0][v] = u; dis[v] = dis[u] + w;
                    st.push_back(v);
                }
        }
    }
    // Replaces v with unique keys + LCAs in DFS order.
    // ans[i] <-> original vertex v[i], i is 0-based.
    // Undirected edges; ans.size() == v.size().
    // Empty keys -> empty tree; otherwise root is 0.
    // Root distances and compressed edge sums must fit in ll.
    // k input keys: O(k log k + k log n) time, O(k) space.
    vector <vector <pil>> getVT(vector <int> &v)
    {
        if (v.empty()) return {};
        auto cmp = [&](int a, int b) {return dfn[a] < dfn[b];};
        sort(v.begin(), v.end(), cmp);
        v.erase(unique(v.begin(), v.end()), v.end());
        int k = (int)v.size();
        for (int i = 1; i < k; ++i)
            v.push_back(lca(v[i - 1], v[i]));
        sort(v.begin(), v.end(), cmp);
        v.erase(unique(v.begin(), v.end()), v.end());
        int m = (int)v.size();
        vector <vector <pil>> ans(m);
        vector <int> st = {0};
        for (int i = 1; i < m; ++i)
        {
            while (!anc(v[st.back()], v[i]))
                st.pop_back();
            int p = st.back();
            ll w = dis[v[i]] - dis[v[p]];
            ans[p].push_back({i, w});
            ans[i].push_back({p, w});
            st.push_back(i);
        }
        return ans;
    }
};
```

1.6 Topological Sorting

```
class Topo
{
    int n;
    vector <int> deg;
    vector <vector <int>> ve;

    public:
    Topo(int n) : n(n), ve(n + 1), deg(n + 1, 0) {}
    void add(int u, int v)
    {
        ve[u].push_back(v);
        ++deg[v];
    }
    vector <int> calc()
    {
        queue <int> q;
        vector <int> lst;
        for (int i = 1; i <= n; ++i)
            if (!deg[i]) q.push(i);
        while (!q.empty())
        {
            int u = q.front(); q.pop();
            lst.push_back(u);
            for (auto v : ve[u])
            {
                --deg[v];
                if (!deg[v]) q.push(v);
            }
        }
        return lst;
    }
};
```

1.7 Shortest Path

1.7.1 Dijkstra

```
class dijkstra
{
    int n, m, cnt;
    vector <int> head, to, nxt, vis;
    vector <ll> dis, val;
    public:
    dijkstra(int n, int m) :
        n(n), m(m), vis(n + 1, 0), head(n + 1, 0), dis(n + 1, 1e18),
        to(2 * m + 1, 0), nxt(2 * m + 1, 0), val(2 * m + 1, 0) {cnt = 0;}

    void add(int u, int v, int w)
    {
        to[++cnt] = v; val[cnt] = w; nxt[cnt] = head[u]; head[u] = cnt;
        to[++cnt] = u; val[cnt] = w; nxt[cnt] = head[v]; head[v] = cnt;
    }
    auto calc(int s)
    {
        priority_queue <pair <ll, int>> q;
        for (int i = 1; i <= n; ++i) vis[i] = 0, dis[i] = INF;
        q.push({0, s});
        dis[s] = 0;
        while (!q.empty())
        {
            int u = q.top().second; q.pop();
            if (vis[u]) continue;
            vis[u] = 1;
            for (int i = head[u]; i; i = nxt[i])
            {
                int v = to[i];
                if (dis[v] > dis[u] + val[i])
                {
                    dis[v] = dis[u] + val[i];
                    q.push({-dis[v], v});
                }
            }
        }
        return dis;
    }
};
```

1.7.2 SPFA

```
class SPFA
{
    int n, m, cnt;
    vector <int> head, to, nxt, val, vis, times;
    vector <ll> dis;
    public:
    SPFA(int n, int m) :
        n(n), m(m), times(n + 1, 0), vis(n + 1, 0), head(n + 1, 0), dis(
      n + 1, INF),
        to(2 * m + 1, 0), nxt(2 * m + 1, 0), val(2 * m + 1, 0) {cnt = 0;}

    void add(int u, int v, int w)
    {
        to[++cnt] = v; val[cnt] = w; nxt[cnt] = head[u]; head[u] = cnt;
        to[++cnt] = u; val[cnt] = w; nxt[cnt] = head[v]; head[v] = cnt;
    }
    auto calc(int s)
    {
        queue <int> q;
        for (int i = 1; i <= n; ++i) vis[i] = 0, dis[i] = INF;
        dis[s] = 0, vis[s] = 1; q.push(s);
        while (!q.empty())
        {
            int u = q.front();
            q.pop(), vis[u] = 0;
            for (int i = head[u]; i; i = nxt[i])
            {
                int v = to[i];
                if (dis[v] > dis[u] + val[i])
                {
                    dis[v] = dis[u] + val[i];
                    times[v] = times[u] + 1;
                    if (times[v] >= n) return {-1}; //Negative Cycle
                    if (!vis[v]) q.push(v), vis[v] = 1;
```

```
                }
            }
        }
        return dis;
    }
};
```

1.8 DCC

```
class DCC
{
    public:
    struct Edge {int to, nxt;};
    vector <Edge> G;
    vector <int> head, dfn, low, col;
    vector <bool> cut;
    stack <int> stk;
    int cnt, cct, n;
    void init(int nn = 0)
    {
        G.assign(2, {0, 0});
        n = nn; head.assign(n + 1, 0);
    }
    void dfs(int u, int f = -1)
    {
         dfn[u] = low[u] = ++cnt;
        int ch = 0;
        for (int i = head[u]; i; i = G[i].nxt)
        if (i != f)
        {
            int v = G[i].to;
            if (dfn[v] < dfn[u]) stk.push(i >> 1);
            if (!dfn[v])
            {
                dfs(v, i ^ 1); ++ch;
                low[u] = min(low[u], low[v]);
                if (low[v] >= dfn[u])
                {
                    int I = 0;
                    ++cct; cut[u] = true;
                    do
                    {
                        assert(!stk.empty());
                        I = stk.top(); stk.pop();
                        col[I] = cct;
                    }
                    while (I != (i >> 1));
                }
            }
            else if (dfn[v] < low[u]) low[u] = dfn[v];
        }
        if (f == -1 && ch == 1) cut[u] = false;
    }
    void tarjan()
    {
        cnt = cct = 0;
        col.assign(G.size() >> 1, 0);
        dfn.assign(n + 1, 0);
        low.assign(n + 1, 0);
        cut.assign(n + 1, false);
        while (!stk.empty()) stk.pop();
        for (int i = 1; i <= n; ++i) if (!dfn[i]) dfs(i);
    }
    void insert(int u, int v)
    {
        G.push_back({v, head[u]}); head[u] = G.size() - 1;
        G.push_back( {u, head[v]}); head[v] = G.size() - 1;
    }
    void insert(vector <vector <int>> &G1)
    {
        for (unsigned u = 1; u < G1.size(); ++u)
            for (int v : G1[u]) insert(u, v);
    }
    bool operator [] (const int &x) const {return cut[x];}
};
```

1.9 Edge-Biconnected Components

```
class EDCC
{
    public:
    struct Edge {int to, nxt;};
    vector <Edge> G;
    vector <int> head, dfn, low, scc;
    vector <bool> cut;
    vector <vector <int>> T;
    stack <int> stk;
    int n, cnt, tot;
    // EDCC g(n); g.insert(u, v); g.init();
    // Vertices, components and inserted edges are 1-based.
    EDCC(int nn = 0)
    {
        n = nn; cnt = tot = 0;
        G.assign(2, {0, 0});
        head.assign(n + 1, 0);
    }
    void insert(int u, int v)
    {
        G.push_back({v, head[u]}); head[u] = G.size() - 1;
        G.push_back({u, head[v]}); head[v] = G.size() - 1;
    }
    void dfs(int u, int f = -1)
    {
        dfn[u] = low[u] = ++cnt;
        stk.push(u);
        for (int i = head[u]; i; i = G[i].nxt)
        if (i != f) // Skip only the reverse parent edge.
        {
            int v = G[i].to;
            if (!dfn[v])
            {
                dfs(v, i ^ 1);
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u]) cut[i >> 1] = true;
            }
            else if (dfn[v] < low[u]) low[u] = dfn[v];
        }
        if (low[u] == dfn[u])
        {
            int v; ++tot;
            do
            {
                v = stk.top(); stk.pop();
                scc[v] = tot;
            }
            while (v != u);
        }
    }
    // O(n + m). T is a forest if the graph is disconnected.
    // scc[u]: component of u; cut[i]: is inserted edge i a bridge?
    void init()
    {
        cnt = tot = 0;
        dfn.assign(n + 1, 0);
        low.assign(n + 1, 0);
        scc.assign(n + 1, 0);
        cut.assign(G.size() >> 1, false);
        while (!stk.empty()) stk.pop();
        for (int i = 1; i <= n; ++i) if (!dfn[i]) dfs(i);
        T.assign(tot + 1, {});
        for (int i = 2; i < (int)G.size(); i += 2)
        if (cut[i >> 1])
        {
            int u = scc[G[i ^ 1].to], v = scc[G[i].to];
            T[u].push_back(v); T[v].push_back(u);
        }
    }
};
```

1.10 Tarjan

```
class Tarjan
{
    int n, m, cnt, times, scc_cnt;
    vector <int> head, to, nxt, low, scc, dfn;
    stack <int> s;
    void tarjan(int u)
    {
        low[u] = dfn[u] = ++times;
        s.push(u);
        for (int i = head[u]; i; i = nxt[i])
        {
            int v = to[i];
            if (!dfn[v])
            {
                tarjan(v);
                low[u] = min(low[u], low[v]);
            }
            else if (!scc[v]) low[u] = min(low[u], dfn[v]);
        }
        if (low[u] == dfn[u])
        {
            ++scc_cnt;
            while (true)
            {
                int x = s.top(); s.pop();
                scc[x] = scc_cnt;
                if (x == u) break;
            }
        }
    }
    public:
    Tarjan(int n, int m) :
```

```
        n(n), m(m), head(n + 2, 0), low(n + 1, 0), dfn(n + 1, 0), scc(n + 1, 0),
        to(2 * m + 2, 0), nxt(2 * m + 2, 0) {cnt = 1; times = scc_cnt = 0;}
    void add(int u, int v) {to[++cnt] = v; nxt[cnt] = head[u]; head[u] = cnt;}
    auto calc()
    {
        for (int i = 1; i <= n; ++i)
            if (!dfn[i]) tarjan(i);
        return scc;
    }
};
```

1.11 Bipartite Graph Matchings

```
class Matching
{
    int l, r; //the number of left/right side points
    vector <vector <int>> ve;
    vector <int> vis, op;
    bool dfs(int u)
    {
        for (auto v : ve[u])
        {
            if (vis[v]) continue;
            vis[v] = 1;
            if (!op[v] || dfs(op[v])) {op[v] = u; return true;}
        }
        return false;
    }
    public:
    Matching(int l, int r) : l(l), r(r), vis(r + 1, 0), op(r + 1, 0), ve(l + 1) {}
    void add(int u, int v) {ve[u].push_back(v);}
    int calc()
    {
        int ans = 0;
        for (int i = 1; i <= l; ++i)
        {
            vis.assign(r + 1, 0);
            if (dfs(i)) ++ans;
        }
        return ans;
    }
};
```

1.12 Flow

1.12.1 Edmonds-Karp

```
template <typename T>
class EK
{
    int n, m, s, t, cnt;
    vector <int> head, to, nxt, vis, pre, edge;
    vector <T> val;
    bool bfs()
    {
        queue <T> q;
        for (int i = 1; i <= n; ++i) vis[i] = 0, pre[i] = edge[i] = -1;
        vis[s] = 1; q.push(s);
        while (!q.empty())
        {
            int u = q.front(); q.pop();
            for (int i = head[u]; i; i = nxt[i])
            {
                int v = to[i];
                if (!vis[v] && val[i])
                {
                    pre[v] = u; edge[v] = i; vis[v] = 1;
                    q.push(v);
                    if (v == t) return 1;
                }
            }
        }
        return 0;
    }
    public :
    EK(int n, int m, int s, int t) :
        n(n), m(m), s(s), t(t),
        vis(n + 1, 0), head(n + 1, 0), pre(n + 1, -1), edge(n + 1, -1),
        to(m + 1, 0), nxt(m + 1, 0), val(m + 1, 0) {cnt = 1;}
    void add(int u, int v, T w)
    {
        to[++cnt] = v; val[cnt] = w; nxt[cnt] = head[u]; head[u] = cnt;
        to[++cnt] = u; val[cnt] = 0; nxt[cnt] = head[v]; head[v] = cnt;
    }
    T calc()
    {
        T ans = 0;
        while (bfs())
        {
            T mn = INF;
            for (int i = t; i != s; i = pre[i]) mn = min(mn, val[edge[i]]);
            for (int i = t; i != s; i = pre[i]) val[edge[i]] -= mn, val[edge[i] ^ 1] += mn;
            ans += mn;
        }
        return ans;
    }
};
```

1.12.2 Dinic

```
template <typename T>
class Dinic
{
    int n, m, s, t, cnt;
    vector <int> head, to, nxt, cur, dep;
    vector <T> val;
    int bfs()
    {
        for (int i = 0; i <= n; ++i) dep[i] = 0, cur[i] = head[i];
        queue <int> q;
        q.push(s), dep[s] = 1;
        while (!q.empty())
        {
            int u = q.front(); q.pop();
            for (int i = head[u]; i; i = nxt[i])
            {
                int v = to[i];
                if (val[i] && !dep[v]) q.push(v), dep[v] = dep[u] + 1;
            }
        }
        return dep[t];
    }
    T dfs(int u, int t, T flow)
    {
        if (u == t) return flow;
        T ans = 0;
        for (int &i = cur[u]; i && ans < flow; i = nxt[i])
        {
            int v = to[i];
            if (val[i] && dep[v] == dep[u] + 1)
            {
                int x = dfs(v, t, min(val[i], flow - ans));
                if (x) val[i] -= x, val[i ^ 1] += x, ans += x;
            }
        }
        if (ans < flow) dep[u] = -1;
        return ans;
    }
    public :
    Dinic(int n, int m, int s, int t) :
        n(n), m(m), s(s), t(t),
        head(n + 1, 0), cur(n + 1, 0), dep(n + 1, 0),
        to(m + 1, 0), nxt(m + 1, 0), val(m + 1, 0) {cnt = 1;}
    void add(int u, int v, T w)
    {
        to[++cnt] = v; val[cnt] = w; nxt[cnt] = head[u]; head[u] = cnt;
        to[++cnt] = u; val[cnt] = 0; nxt[cnt] = head[v]; head[v] = cnt;
    }
    T calc()
    {
        T ans = 0;
        while (bfs())
        {
            T x;
            while ((x = dfs(s, t, INF))) ans += x;
        }
        return ans;
    }
};
```

```
class Dinic
{
    typedef long long ll;
    static const int N=1e6+10, M=4e6+10;
    static const ll INF=4e18;
    int cgt, head[N], des[M], nxt[M], dep[N], que[N], H, T, h2[N];
    ll cap[M];
    inline ll dfs(const int &u, ll in=INF)
    {
        if(u==t)return in; ll out=0;
        for(int &i=h2[u]; i; i=nxt[i])
        {
            int v=des[i];
            if(cap[i]&&dep[v]==dep[u]+1)
            {
                ll tmp=dfs(v, min(in, cap[i]));
```

```
                out+=tmp; cap[i]-=tmp; cap[i^1]+=tmp;
                if(!(in-=tmp))break;
            }
        }
        return out?out:dep[u]=0;
    }
    inline bool bfs()
    {
        memset(dep+1, 0, n*sizeof(ll));
        dep[que[H=T=0]=s]=1;
        while(H<=T)
        {
            int u=que[H++];
            for(int i=head[u]; i; i=nxt[i])if(cap[i])
            {
                int v=des[i];
                if(!dep[v])dep[que[++T]=v]=dep[u]+1;
            }
        }
        return dep[t];
    }
    public:
        int n, s, t;
        inline void Init(const int &sz=0)
        {
            memset(head+1, 0, n*sizeof(ll));
            cgt=1; n=sz;
        }
        inline ll Max_Flow(const int &ss, const int &tt)
        {
            s=ss; t=tt; ll ans=0;
            while(bfs())
            {
                memcpy(h2+1, head+1, n*sizeof(ll));
                ans+=dfs(s, INF);
            }
            return ans;
        }
        inline bool ins(const int &x, const int &y, const ll &w)
        {
            if(x>n||y>n||x<1||y<1)return false;
            nxt[++cgt]=head[x]; des[head[x]=cgt]=y; cap[cgt]=w;
            nxt[++cgt]=head[y]; des[head[y]=cgt]=x; cap[cgt]=0;
            return true;
        }
        inline int newNode(){return ++n;}
}a;


namespace MCMF
{
    typedef long long Val_Type; typedef long long Cap_Type;
    const Val_Type INF_v=4e18; const Cap_Type INF_c=4e18;
    const int N=1e6+10, M=2e6+10; int head[N], des[M], nxt[M], cgt=1;
      Cap_Type cap[M]; Val_Type val[M];
    template<typename T> inline T Abs(T x){return x<0?-x:x;}
    inline void ins(const int &x, const int &y, const int &c, const
      Val_Type &w)
    {
        nxt[++cgt]=head[x]; des[head[x]=cgt]=y; cap[cgt]=c; val[cgt]=w;
        nxt[++cgt]=head[y]; des[head[y]=cgt]=x; cap[cgt]=0; val[cgt]=-w;
    }
    int que[20000010], H, T; Val_Type dis[N], mnc; int s, t, S; bool inq
      [N];
    inline bool spfa()
    {
        for(int i=0; i<=S+2; ++i)dis[i]=INF_v; dis[que[H=T=0]=s]=0;
        while(H<=T)
        {
            int u=que[H++]; inq[u]=false; for(int i(head[u]); i; i=nxt[i
      ])
                {int v=des[i]; if(cap[i]&&dis[v]>dis[u]+val[i]){dis[v]=
      dis[u]+val[i]; if(!inq[v])inq[que[++T]=v]=true;}}
        }
        return dis[t]<INF_v;
    }
    inline Cap_Type dfs(const int &u=s, Cap_Type in=INF_c)
    {
        if(u==t)return in; Cap_Type out=0; inq[u]=true;
        for(int i(head[u]); i; i=nxt[i])
        {
            int v=des[i]; if(cap[i]&&!inq[v]&&dis[v]==(dis[u]+val[i]))
            {
                Cap_Type tmp=dfs(v, cap[i]<in?cap[i]:in); mnc+=val[i]*
      tmp;
                cap[i]-=tmp; cap[i^1]+=tmp; out+=tmp; if(!(in-=tmp))
      break;
            }
        }
        if(!out)dis[u]=4e18; inq[u]=false; return out;
    }
    inline Cap_Type dinic(){Cap_Type res=0; while(spfa())res+=dfs();
      return res;}
    inline void Clear(){memset(head, 0, (S+2)*sizeof(head[0])); cgt=1; S
      =0; mnc=0;}
}
```

```
using MCMF::ins; using MCMF::s; using MCMF::t; using MCMF::S; using MCMF:
      :dinic; using MCMF::mnc;
```

2 Data Structure

2.1 Segment Tree

```
namespace Seg
{
    struct SEG{int l,r,sum,tag,vl,vr;}t[N];int Rt,cnt;
    inline void update(int k){t[k].sum=t[t[k].l].sum+t[t[k].r].sum;}
    inline void pushdown(int k)
    {
        if(t[k].l)t[t[k].l].sum+=t[k].tag*(t[t[k].l].vr-t[t[k].l].vl+1),
      t[t[k].l].tag+=t[k].tag;
        if(t[k].r)t[t[k].r].sum+=t[k].tag*(t[t[k].r].vr-t[t[k].r].vl+1),
      t[t[k].r].tag+=t[k].tag;t[k].tag=0;
    }
    void add(int cur,int ql,int qr,int val,int l,int r)
    {
        int l=t[cur].vl,r=t[cur].vr,mid=(l+r)>>1;
        if(ql<=l&&r<=qr){t[cur].sum+=val*(r-l+1);t[cur].tag+=val;return;}

        pushdown(cur);if(ql<=mid)add(t[cur].l,ql,qr,val,l,mid);
        if(qr>mid)add(t[cur].r,ql,qr,val,mid+1,r);update(cur);
    }
    int query(int cur,int ql,int qr,int l,int r)
    {
        int res=0,l=t[cur].vl,r=t[cur].vr,mid=(l+r)>>1;
        pushdown(cur);if(ql<=l&&r<=qr){return t[cur].sum;}
        if(ql<=mid)res+=query(t[cur].l,ql,qr,l,mid);
        if(qr>mid)res+=query(t[cur].r,ql,qr,mid+1,r);update(cur);return
      res;
    }
    void build(int l,int r,int &cur)
    {
        t[cur=++cnt].vl=l;t[cur].vr=r;if(l==r){t[cur].sum=a[l];return;}
        int mid=(l+r)>>1;if(l<=mid)build(l,mid,t[cur].l);
        if(r>mid)build(mid+1,r,t[cur].r);update(cur);
    }
    int find(int cur,int ql,int qr,int x,int l,int r)
    {
        int l=t[cur].vl,r=t[cur].vr,mid=(l+r)>>1;
        if(ql<=l&&r<=qr)
        {
            if(t[cur].sum<x)return -1;
            if(l==r)return l;
            pushdown(cur);
            if(t[t[cur].l].sum>=x)return find(t[cur].l,ql,qr,x,l,mid);
            return find(t[cur].r,ql,qr,x-t[t[cur].l].sum,mid+1,r);
        }
        pushdown(cur);
        if(qr<=mid)return find(t[cur].l,ql,qr,x,l,mid);
        if(ql>mid)return find(t[cur].r,ql,qr,x,mid+1,r);
        int res=find(t[cur].l,ql,qr,x,l,mid);
        if(res!=-1)return res;
        int sum=query(t[cur].l,ql,mid,l,mid);
        return find(t[cur].r,ql,qr,x-sum,mid+1,r);
    }
    inline int merge(int x,int y,int l,int r)
    {
        if(!x||!y)return x|y;
        if(l==r){t[x].sm+=t[y].sm;return x;}
        int mid=(l+r)>>1;
        t[x].l=merge(t[x].l,t[y].l,l,mid);
        t[x].r=merge(t[x].r,t[y].r,mid+1,r);
        update(x);return x;
    }
    //Persistent
    inline void merge(int &x,int y,int l=1,int r=N-10)
    {
        if(!x||!y){x+=y;return;}
        int nx=++cnt;t[nx]=t[x];x=nx;
        if(l==r){t[x].mx+=t[y].mx;t[x].res=t[x].mx?l:0;return;}
        int mid=(l+r)>>1;
        merge(t[x].l,t[y].l,l,mid);merge(t[x].r,t[y].r,mid+1,r);
        update(x);
    }
}
```

2.2 Fenwick Tree

```
inline void add(int *t,int p,int v){while(p<=n)t[p]+=v,p+=(p&-p);}
inline int qry(int *t,int p){int res=0;while(p)res+=t[p],p^=(p&-p);
      return res;}
inline int qry(int *t,int k)//return n+1 if all satisfy
{
    int p=0,tot=0;
    for(int i=20;~i;--i)if((p|1<<i)<=n&&tot+t[p|(1<<i)]<k)tot+=t[p|=(1<<
      i)];
    return p+1;
}
```

2.3 Bitset

```
class Bitset{
    int n,m;
    vector<uint64_t>a;

    static inline uint64_t rev(uint64_t x){
        x=((x&0x5555555555555555ULL)<<1)|((x>>1)&0x5555555555555555ULL);
        x=((x&0x3333333333333333ULL)<<2)|((x>>2)&0x3333333333333333ULL);
        x=((x&0x0f0f0f0f0f0f0f0fULL)<<4)|((x>>4)&0x0f0f0f0f0f0f0f0fULL);
        x=((x&0x00ff00ff00ff00ffULL)<<8)|((x>>8)&0x00ff00ff00ff00ffULL);
        x=((x&0x0000ffff0000ffffULL)<<16)|((x>>16)&0x0000ffff0000ffffULL)
      ;
        return (x<<32)|(x>>32);
    }

    inline void fix(){if(n&63)a.back()&=((1ULL<<(n&63))-1);}
    //set i>=n to zero

public:
    class Ref{
        uint64_t&x,m;
    public:
        Ref(uint64_t&x,uint64_t m):x(x),m(m){}
        Ref&operator=(bool v){if(v)x|=m;else x&=~m;return *this;}
        Ref&operator=(const Ref&v){return *this=bool(v);}
        operator bool()const{return x&m;}
        Ref&flip(){x^=m;return *this;}
    };

    Bitset(int n=0):n(n),m((n+63)>>6),a(m){}

    Ref operator[](int p){return Ref(a[p>>6],1ULL<<(p&63));}
    bool operator[](int p)const{return (a[p>>6]>>(p&63))&1;}
    bool test(int p)const{return (*this)[p];}

    Bitset&set(){
        fill(a.begin(),a.end(),~0ULL);
        fix();return *this;
    }
    Bitset&set(int p,bool v=1){(*this)[p]=v;return *this;}
    Bitset&reset(){fill(a.begin(),a.end(),0);return *this;}
    Bitset&reset(int p){(*this)[p]=0;return *this;}

    Bitset&flip(){
        for(auto&x:a)x=~x;
        fix();return *this;
    }
    Bitset&flip(int p){a[p>>6]^=1ULL<<(p&63);return *this;}

    bool any()const{
        for(auto x:a)if(x)return 1;
        return 0;
    }
    bool none()const{return !any();}

    int count()const{
        int s=0;
        for(auto x:a)s+=__builtin_popcountll(x);
        return s;
    }

    int size()const{return n;}

    Bitset&operator&=(const Bitset&b){
        for(int i=0;i<m;i++)a[i]&=b.a[i];
        return *this;
    }
    Bitset&operator|=(const Bitset&b){
        for(int i=0;i<m;i++)a[i]|=b.a[i];
        return *this;
    }
    Bitset&operator^=(const Bitset&b){
        for(int i=0;i<m;i++)a[i]^=b.a[i];
        return *this;
    }

    Bitset&operator<<=(int k){
        if(k>=n)return reset();
        int q=k>>6,r=k&63;
        if(q){
            for(int i=m-1;i>=q;i--)a[i]=a[i-q];
            for(int i=0;i<q;i++)a[i]=0;
        }
        if(r){
            for(int i=m-1;i;i--)a[i]=(a[i]<<r)|(a[i-1]>>(64-r));
            a[0]<<=r;
        }
        fix();return *this;
    }

    Bitset&operator>>=(int k){
        if(k>=n)return reset();
        int q=k>>6,r=k&63;
        if(q){
```

```
            for(int i=0;i+q<m;i++)a[i]=a[i+q];
            for(int i=m-q;i<m;i++)a[i]=0;
        }
        if(r){
            for(int i=0;i+1<m;i++)a[i]=(a[i]>>r)|(a[i+1]<<(64-r));
            a[m-1]>>=r;
        }
        return *this;
    }

    Bitset&reverse(){
        for(int i=0,j=m-1;i<=j;i++,j--){
            uint64_t x=rev(a[i]);
            a[i]=rev(a[j]);
            a[j]=x;
        }
        if(n&63){
            int r=n&63,s=64-r;
            for(int i=0;i+1<m;i++)a[i]=(a[i]>>s)|(a[i+1]<<r);
            a[m-1]>>=s;
        }
        fix();return *this;
    }
    Bitset&operator+=(const Bitset&b){
        uint64_t c=0;
        for(int i=0;i<m;i++){
            uint64_t x=a[i],y=b.a[i];
            a[i]=x+y+c;
            c=(a[i]<x)||(c&&a[i]==x);
        }
        fix();
        return *this;
    }

    Bitset&operator-=(const Bitset&b){
        uint64_t c=0;
        for(int i=0;i<m;i++){
            uint64_t x=a[i],y=b.a[i];
            a[i]=x-y-c;
            c=(x<y)||(c&&x==y);
        }
        fix();
        return *this;
    }
    friend Bitset operator&(Bitset x,const Bitset&y){return x&=y;}
    friend Bitset operator|(Bitset x,const Bitset&y){return x|=y;}
    friend Bitset operator^(Bitset x,const Bitset&y){return x^=y;}
    friend Bitset operator<<(Bitset x,int k){return x<<=k;}
    friend Bitset operator>>(Bitset x,int k){return x>>=k;}
    friend Bitset operator~(Bitset x){return x.flip();}
};
/*
_Find_first() : the first 1
_Find_next(p) : the position of the first 1 after p (not included p)
*/
```

2.4 Heavy-Light Decomposition

```
class HLD
{
    public:
        vector<vector<int> > G;
        int n, rt, cdt;
        HLD() : n(0), rt(1), cdt(0) {}
        vector<int> top,dep,fa,mxs,siz,dfn;
        void dfs1(int u, int f)
        {
            siz[u] = 1;
            fa[u] = f;
            dep[u] = dep[f] + 1;
            int mx = 0;
            for(int v: G[u]) if(v != f)
            {
                dfs1(v, u);
                siz[u] += siz[v];
                if(siz[v] > mx)
                {
                    mx = siz[v];
                    mxs[u] = v;
                }
            }
        }
        void dfs2(int u, int f, int tp)
        {
            if(mxs[u]) dfs2(mxs[u], u, tp);
            top[u] = tp;
            dfn[u] = ++cdt;
            // subtree : [dfn[u], dfn[u] + siz[u] - 1]
            // chain : [dfn[top[u]], dfn[u]]
            for(int v: G[u]) if(v != f && v != mxs[u])
            {
                dfs2(v, u, v);
            }
        }

        HLD(vector<vector<int> > &T, int RT = 1)
        {
            G = T;
            rt = RT;
            n = int(T.size()) - 1;
            top.assign(n+1, 0);
            dep.assign(n+1, 0);
            fa.assign(n+1, 0);
            mxs.assign(n+1, 0);
            siz.assign(n+1, 0);
            dfn.assign(n+1, 0);
            dfs1(rt, 0);
            cdt = 0;
            dfs2(rt, 0, rt);
        }
        int LCA(int u, int v)
        {
            while(top[u] != top[v])
            {
                dep[top[u]] > dep[top[v]] ? u = fa[top[u]] : v = fa[top[
    v]];
            }
            return dep[u] > dep[v] ? v : u;
        }
};
```

2.5 Longest Chain Decomposition

```
//Total Length of Chains is O(n)
//Each merge will eliminate 1 node of the shorter chains
const int N = 1e6 + 10;
long long F[N];//f[rt][i]: the sum of nodes at distance exactly i from
     rt
//e.g. the sum of the nodes with distance =i
int mxs[N], dep[N], mxd[N];
long long a[N];
vector<int> G[N];
int n, rt;
void dfs(int u = 1, int fa = 0)
{
    dep[u] = dep[fa] + 1;
    int mx = -1;
    mxd[u] = dep[u];
    for(int v: G[u]) if(v != fa)
    {
        dfs(v, u);
        mxd[u] = max(mxd[u], mxd[v]);
        if(mxd[v] > mx)
        {
            mx = mxd[v];
            mxs[u] = v;
        }
    }
}
void DP(int u = 1, int fa = 0, long long *f = F)
{
    f[0] += a[u];
    long long *tmp = f + (mxd[u] - dep[u]) + 1;
    for(int v : G[u]) if(v != fa)
    {
        if(v != mxs[u])
        {
            long long *fv = tmp;
            DP(v, u, fv);
            for(int i = 0; i <= mxd[v] - dep[v]; ++i)
            {
                f[i + 1] += fv[i];
                fv[i] = 0;
            }
        }
        else
        {
            DP(v, u, f + 1);
        }
    }
}
```

2.6 Link Cut Tree

```
const int N = 1e5 + 10;
struct Splay {
    int s[2] = {0, 0};
    int fa = 0;
    int sum = 0;
    int vl = 0;
    bool tg = false;
} t[N];
int n;
```

```
// Check whether x is not the root of its auxiliary tree.
// isSon(x) == false : is Root
inline bool isSon(const int &x) {return t[t[x].fa].s[0] == x || t[t[x].fa].s[1] == x;}
inline void rev(const int &x) {if (!x) return;swap(t[x].s[0], t[x].s[1]);t[x].tg ^= 1;}
inline void update(const int &x) {t[x].sum = t[t[x].s[0]].sum ^ t[t[x].s[1]].sum ^ t[x].vl;}
inline void pushdown(const int &x) {if (!t[x].tg) return;t[x].tg = false;rev(t[x].s[0]);rev(t[x].s[1]);}

// Rotate x in its auxiliary tree.
inline void rotate(const int &x) {
    int y = t[x].fa;
    int z = t[y].fa;
    bool k = (t[y].s[1] == x);
    if (isSon(y)) t[z].s[t[z].s[1] == y] = x;
    int w = t[x].s[k ^ 1];
    t[x].s[k ^ 1] = y;
    t[y].s[k] = w;
    if (w) t[w].fa = y;
    t[x].fa = z;
    t[y].fa = x;
    update(y);
    update(x);
}
int sk[N], tp;
// Splay x and push all lazy tags on the path to x.
inline void splay(const int x) {
    tp = 1;
    sk[tp] = x;
    while (isSon(sk[tp])) {
        ++tp;
        sk[tp] = t[sk[tp - 1]].fa;
    }
    while (tp) pushdown(sk[tp--]);
    while (isSon(x)) {
        int y = t[x].fa;
        if (isSon(y)) {
            bool zigZig = (t[y].s[0] == x) == (t[t[y].fa].s[0] == y);
            rotate(zigZig ? y : x);
        }
        rotate(x);
    }
    update(x);
}

// Expose the preferred path from x to the origin-tree root.
inline void access(int x) {
    for (int y = 0; x; x = t[y = x].fa) {
        splay(x);
        t[x].s[1] = y;
        update(x);
    }
}

// Make x the root of its original tree.
inline void makeRt(const int &x) {
    access(x);
    splay(x);
    rev(x);
}

// Find the represented-tree root containing x.
inline int fndRt(int x) {
    access(x);
    splay(x);
    while (t[x].s[0]) {
        pushdown(x);
        x = t[x].s[0];
    }
    splay(x);
    return x;
}

// Expose the path from x to y in y's auxiliary tree. (Exactly x->y, Rooted at y)
inline void split(const int &x, const int &y) {
    makeRt(x);
    access(y);
    splay(y);
}

// Link the trees containing x and y.
inline void lnk(const int &x, const int &y) {
    makeRt(x);
    if (fndRt(y) != x) t[x].fa = y;
}

// Cut the edge x-y if it exists.
inline void cut(const int &x, const int &y) {
    makeRt(x);
    if (fndRt(y) == x && t[y].fa == x && !t[y].s[0]) {
        t[y].fa = t[x].s[1] = 0;
        update(x);
    }
}

// Modify the value stored at x.
inline void modify(const int &x, const int &value) {
    access(x);
    splay(x);
    t[x].vl = value;
    update(x);
}
```

2.7 Treap

```
namespace VAL
{
    inline void update(int k){t[k].siz=t[t[k].l].siz+t[t[k].r].siz+1;}
    inline int new_node(int k){t[++cnt].val=k;t[cnt].siz=1;t[cnt].key=rand();return cnt;}
    inline void split(int k,int cur,int &x,int &y)
    {
        if(!cur){x=y=0;return;}pushdown(cur);
        else if(t[cur].val<=k)x=cur,split(k,t[cur].r,t[cur].r,y);
        else y=cur,split(k,t[cur].l,x,t[cur].l);update(cur);
    }
    inline int merge(int L,int R)
    {
        if(!L||!R)return L+R;
        pushdown(L);pushdown(R);
        if(t[L].key<t[R].key)
        {
            t[L].r=merge(t[L].r,R);
            update(L);return L;
        }
        else
        {
            t[R].l=merge(L,t[R].l);
            update(R);return R;
        }
    }
}
namespace SIZ
{
    inline void split(int k,int cur,int &x,int &y)
    {
        if(!cur){x=y=0;return;}pushdown(cur);
        if(k<=t[t[cur].l].siz){split(k,t[cur].l,x,t[y=cur].l);update(cur);return;}
        else {split(k-t[t[cur].l].siz-1,t[cur].r,t[x=cur].r,y);update(cur);return;}
    }
    inline int merge(int L,int R)
    {
        if(!L||!R)return L+R;pushdown(L);pushdown(R);
        if(t[L].key<t[R].key){t[L].r=merge(t[L].r,R);update(L);return L;}
        else{t[R].l=merge(L,t[R].l);update(R);return R;}
    }
    inline int rnk(int k){int res=t[t[k].l].siz;while(t[k].f){res+=(k==t[t[k].f].r)?
        t[t[t[k].f].l].siz+1:0;k=t[k].f;}return res+1;}
}
namespace MERGE//merge same values into one node
{
    struct Node
    {
        int l,r,f,ct,sz,vl;
        Node(){l=r=vl=sz;}
        Node(ll v,ll c){l=r=f;ct=sz=c;vl=v;}
    }t[N];
    int n,stk[N],top,cnt,T1,T2,T3,T4,T5,Rt;
    inline int node(ll v,ll c)
    {
        int u=top?stk[top--]:++cnt;
        t[u]=Node(v,c);return u;
    }
    inline int copy(int x)
    {
        int u=top?stk[top--]:++cnt;
        t[u]=t[x];return u;
    }
    inline void update(int x)
    {
        pushdown(x);
        t[x].sz=t[t[x].l].sz+t[t[x].r].sz+t[x].ct;
        if(t[x].l)t[t[x].l].f=x;
        if(t[x].r)t[t[x].r].f=x;
        t[x].f=0;
    }
    inline void split(int cur,int k,int &x,int &y)
    {
        if(!cur){x=y=0;return;}
        pushdown(cur);
        if(t[t[cur].l].sz>=k)
        {
```

```
            y=cur;
            split(t[cur].l,k,x,t[y].l);
            update(y);
        }
        else if(t[t[cur].l].sz+t[cur].ct>=k)
        {
            k-=t[t[cur].l].sz;
            if(k==t[cur].ct)
            {
                y=t[cur].r;
            }
            else
            {
                y=copy(cur);
                t[y].ct-=k;
                t[y].l=0;
            }
            x=cur;t[x].ct=k;
            t[x].r=0;
            update(x);
            update(y);
        }
        else
        {
            x=cur;
            split(t[cur].r,k-t[t[cur].l].sz-t[cur].ct,t[x].r,y);
            update(x);
        }
    }
    inline int merge(int x,int y)
    {
        if(!x||!y)return x|y;
        pushdown(x);pushdown(y);
        if(int(rnd())%(t[x].sz+t[y].sz)<t[x].sz)
        {
            t[x].r=merge(t[x].r,y);
            update(x);
            return x;
        }
        else
        {
            t[y].l=merge(x,t[y].l);
            update(y);
            return y;
        }
    }
}
```

2.8 Cartesian Tree

```
//heap-order(root is max) on values and BST-order on keys
stack <int> s;
vector <int> L(n + 1, 0), R(n + 1, 0);
vector <int> ql(n + 1, 0), qr(n + 1, 0);
for (int i = 1; i <= n; ++i)
{
    int la = -1;
    while (!s.empty() && a[s.top()] < a[i]) la = s.top(), s.pop();
    if (!s.empty()) R[s.top()] = i;
    if (la != -1) L[i] = la;
    s.push(i);
}
int Rt = 0;
while (!s.empty()) Rt = s.top(), s.pop();
auto dfs = [&](auto self, int u, int l, int r)->void
{
    ql[u] = l; qr[u] = r;
    if(L[u])self(self, L[u], l, u - 1);
    if(R[u])self(self, R[u], u + 1, r);
};
dfs(dfs, Rt, 1, n);
```

2.9 Sparse Table

```
vector <int> lg(n + 1, -1);
vector <vector <int>> f(20, vector <int> (n + 1, INF));
for (int i = 1; i <= n; ++i) lg[i] = lg[i >> 1] + 1;
for (int i = 1; i <= n; ++i) f[0][i] = a[i];
for (int j = 1; j < 20; ++j)
    for (int i = 1; i + (1 << j) - 1 <= n; ++i) f[j][i] = min(f[j - 1][i
      ], f[j - 1][i + (1 << (j - 1))]);
auto query = [&] (int l, int r) -> int {return min(f[lg[r - l + 1]][l],
      f[lg[r - l + 1]][r - (1 << lg[r - l + 1]) + 1]);};
```

2.10 Persistent Data Structure

2.10.1 Persistent Segment Tree

```
struct Seg{int l,r,val;}t[N*64]; int cnt,a[N],Rt[N],dep[N],Mx[N];
inline void build(int &x,int l,int r)
{
    int mid=(l+r)>>1;x=++cnt;if(l==r){t[x].val=a[l];return;}
    build(t[x].l,l,mid);build(t[x].r,mid+1,r);return;
}
inline void Fix(int cur,int &x,int loc,int val,int l,int r)
{
    t[x=++cnt]=t[cur];int mid=(l+r)>>1;
    if(l==r){t[x].val=val;dep[x]=dep[cur];return;}
    if(loc<=mid)Fix(t[cur].l,t[x].l,loc,val,l,mid);
    else Fix(t[cur].r,t[x].r,loc,val,mid+1,r);
}
inline int Query(int cur,int loc,int l,int r)
{
    int mid=(l+r)>>1;if(l==r)return t[cur].val;
    if(loc<=mid)return Query(t[cur].l,loc,l,mid);
    return Query(t[cur].r,loc,mid+1,r);
}
```

2.10.2 Persistent Treap

```
namespace PFHQ//Interval Copy
{
    #include<bits/stdc++.h>
    #define rep(a,b,c) for(register int c=(a);c<=(b);++c)
    using namespace std;
    const int Mod=1e9+7,INF=0x3f3f3f3f,N=4e6+5;
    inline int read()
    {
        int res=0;char ch=getchar();while(ch<'0'||ch>'9')ch=getchar();
        while(ch<='9'&&ch>='0')res=res*10+(ch^48),ch=getchar();return
      res;
    }
    inline int add(int &x,int y){x=((x+y>=Mod)?(x+y-Mod):(x+y));}
    struct FHQ{int l,r,siz,val,sum,tag[3];}t[N];
    int a[N],cnt,Root,top,n,m,T1,T2,T3,T4,T5;
    inline int Node(int val)
    {
        t[++cnt].siz=1;t[cnt].sum=t[cnt].val=val;t[cnt].tag[0]=INF;
        t[cnt].l=t[cnt].r=t[cnt].tag[1]=t[cnt].tag[2]=0;return cnt;
    }
    inline int copy(int x){t[++cnt]=t[x];return cnt;}
    inline void update(int x)
    {
        t[x].siz=t[t[x].l].siz+t[t[x].r].siz+1;
        t[x].sum=((t[t[x].l].sum+t[t[x].r].sum)%Mod+t[x].val)%Mod;
    }
    inline void push2(int x){swap(t[x].l,t[x].r);t[x].tag[2]^=1;}
    inline void push1(int x,int k){add(t[x].val,k);add(t[x].tag[1],k);t[
      x].sum=
    (t[x].sum+1ll*t[x].siz*k%Mod)%Mod;}
    inline void push0(int x,int k)
    {t[x].tag[1]=0;t[x].tag[0]=t[x].val=k;t[x].sum=1ll*t[x].siz*k%Mod;}
    inline void pushdown(int cur)
    {
        if(t[cur].tag[0]!=INF||t[cur].tag[1]||t[cur].tag[2])
        {
            if(t[cur].l) t[cur].l = copy(t[cur].l);
            if(t[cur].r) t[cur].r = copy(t[cur].r);
        }
        if(t[cur].tag[0]!=INF)
        {
            if(t[cur].l) push0(t[cur].l,t[cur].tag[0]);
            if(t[cur].r) push0(t[cur].r,t[cur].tag[0]);
            t[cur].tag[0] = INF;
        }
        if(t[cur].tag[1])
        {
            if(t[cur].l) push1(t[cur].l,t[cur].tag[1]);
            if(t[cur].r) push1(t[cur].r,t[cur].tag[1]);
            t[cur].tag[1] = 0;
        }
        if(t[cur].tag[2]){if(t[cur].l)push2(t[cur].l);if(t[cur].r)push2(
      t[cur].r);t[cur].tag[2]=0;}
    }
    inline int merge(int x,int y)
    {
        if(!x||!y)return x+y;pushdown(x);pushdown(y);
        if(rand()%(t[x].siz+t[y].siz)<t[x].siz){x=copy(x);t[x].r=merge(t
      [x].r,y);update(x);return x;}
        y=copy(y);t[y].l=merge(x,t[y].l);update(y);return y;
    }
    inline void split(int cur,int k,int &x,int &y)
    {
        if(!cur){x=y=0;return;} pushdown(cur);
        if(t[t[cur].l].siz>=k)y=copy(cur),split(t[y].l,k,x,t[y].l),
      update(y);
        else x=copy(cur),split(t[x].r,k-t[t[cur].l].siz-1,t[x].r,y),
      update(x);
    }
    inline void build(int l,int r,int &cur)
    {
```

```
        int mid=(l+r)>>1;cur=Node(a[mid]);
        if(l<mid)build(l,mid-1,t[cur].l);
        if(r>mid)build(mid+1,r,t[cur].r);update(cur);
    }
    inline void change(int l,int r,int k)
    {
        split(Root,l-1,T1,T2);split(T2,r-l+1,T2,T3);
        T2=copy(T2);push0(T2,k);Root=merge(merge(T1,T2),T3);
    }
    inline void modify(int l,int r,int k)
    {
        split(Root,l-1,T1,T2);split(T2,r-l+1,T2,T3);
        T2=copy(T2);push1(T2,k);Root=merge(merge(T1,T2),T3);
    }
    inline void Copy(int l1,int r1,int l2,int r2)//swap
    {
        if(l1>l2)swap(l1,l2),swap(r1,r2);
        split(Root,l1-1,T1,T2);split(T2,r1-l1+1,T2,T3);
        split(T3,l2-r1-1,T3,T4);split(T4,r2-l2+1,T4,T5);
        Root=merge(merge(merge(merge(T1,T4),T3),T2),T5);
    }
    inline void paste(int l1,int r1,int l2,int r2)//copy
    {
        bool flag = 0;if(l1>l2)swap(l1,l2),swap(r1,r2),flag=1;
        split(Root,l1-1,T1,T2);split(T2,r1-l1+1,T2,T3);
        split(T3,l2-r1-1,T3,T4);split(T4,r2-l2+1,T4,T5);
        if(flag) T2=copy(T4);else T4=copy(T2);
        Root=merge(merge(merge(merge(T1,T2),T3),T4),T5);
    }
    inline int query(int l,int r)
    {
        split(Root,l-1,T1,T2);split(T2,r-l+1,T2,T3);
        int res=t[T2].sum;Root=merge(merge(T1,T2),T3);return res;
    }
    inline void dfs(int cur)
    {
        pushdown(cur);if(t[cur].l)dfs(t[cur].l);
        a[++top]=t[cur].val;if(t[cur].r)dfs(t[cur].r);
    }
    inline void reverse(int l,int r)
    {
        split(Root,l-1,T1,T2);split(T2,r-l+1,T2,T3);
        T2=copy(T2);push2(T2);Root=merge(merge(T1,T2),T3);
    }
    int main()
    {
        srand(time(0));
        n=read();m=read();rep(1,n,i)a[i]=read();build(1,n,Root);rep(1,m,
      i)
        {
            int opt=read(),l1=read(),r1=read(),l2,r2;
            switch(opt)
            {
                case 1: printf("%d\n",query(l1,r1));break;
                case 2: change(l1,r1,read());break;
                case 3: modify(l1,r1,read());break;
                case 4: l2=read();r2=read();paste(l1,r1,l2,r2);break;
                case 5: l2=read();r2=read();Copy(l1,r1,l2,r2);break;//
      swap
                case 6: reverse(l1,r1);break;
            }
            if(cnt>N-1000000)dfs(Root),top=Root=cnt=0,build(1,n,Root);
        }
        top=0;dfs(Root);rep(1,n,i)printf("%d ",a[i]);return 0;
    }
}
```

2.11 Linear-Basis

```
template <typename T>
class Basis
{
    int n, lg;
    vector <T> p, ex, nw; vector <vector <int>> id;
    public :
    Basis(int n, int lg) : n(n), lg(lg), p(lg + 1, 0), nw(lg + 1, 0), id
      (lg + 1) {}
    void modify(T x, int idx)
    {
        vector <int> cur(n + 1, 0); cur[idx] = 1;
        for (int i = lg; ~i; --i)
        {
            if (!((1ll << i) & x)) continue;
            if (!p[i]) {p[i] = x; id[i] = cur; break;}
            else
            {
                x ^= p[i];
                for (int j = 1; j <= n; ++j) cur[j] ^= id[i][j];
            }
        }
    }
    T get_max()
    {
        T ans = 0;
        for (int i = lg; ~i; --i)
            if ((ans ^ p[i]) > ans) ans ^= p[i];
        return ans;
    }
    T get_min()
    {
        T ans = 0; int cnt = 0;
        for (int i = 0; i <= lg; ++i) cnt += p[i] > 0;
        if (cnt < n) return 0;
        for (int i = 0; i <= lg; ++i)
            if (p[i] > 0) return p[i];
    }
    void change()
    {
        for (int i = 0; i <= lg; ++i) nw[i] = p[i];
        for (int i = 0; i <= lg; ++i)
            for (int j = i - 1; ~j; --j)
                if (nw[i] >> j & 1) nw[i] ^= nw[j];
        for (int i = 0; i <= lg; ++i)
            if (nw[i]) ex.push_back(i);
    }
    T get_min_k(T k)
    {
        int sz = (int)ex.size();
        if (sz < n) --k; // element 0
        if (k > (1ll << sz) - 1) return -1;
        T ans = 0;
        for (int j = 0; j < sz; ++j)
            if (k >> j & 1) ans ^= nw[ex[j]];
        return ans;
    }
    bool find(T x)
    {
        for (int i = lg; ~i; --i)
        {
            if (!(x >> i & 1)) continue;
            if (!p[i]) return false;
            x ^= p[i];
        }
        return true;
    }
    vector <int> getid(T x) // if find (x) == 1
    {
        vector <int> res(n + 1, 0);
        for (int i = lg; ~i; --i)
        {
            if (!(x >> i & 1)) continue;
            x ^= p[i];
            for (int j = 1; j <= n; ++j) res[j] ^= id[i][j];
        }
        return res;
    }
};
```

3 Math

3.1 Subset Enumeration

```
for (int S = 1; S  < (1 << n); ++S)
    for (int S0 = S; S0; S0 = (S0 - 1) & S)
```

3.2 Elementary Arithmetic Bucket

```
template <typename T, int MOD = 1000000007>
class Z
{
    T x;
    Z <T, MOD> qpow(Z x, T y)
    {
        Z <T, MOD> res(1);
        while (y)
        {
            if (y & 1) res *= x;
            x *= x;
            y >>= 1;
        }
        return res;
    }
    public :
    Z() : x(0) {}
    Z(T x) : x(x) {}
    T get() {return x;}
    Z & operator = (T o) {x = o % MOD; return *this;}
    Z & operator += (Z o) {x = (x + o.x + MOD) % MOD; return *this;}
    Z & operator -= (Z o) {x = (x - o.x + MOD) % MOD; return *this;}
    Z & operator *= (Z o) {x = 1ll * o.x * x % MOD; return *this;}
    Z & operator ^= (Z o) {x = qpow(x, o.x).x; return *this;}
    Z & operator /= (Z o) {*this *= o ^ (MOD - 2); return *this;}
    Z & operator %= (Z o) {x = x % o.x; return *this;}
    friend Z operator + (Z x, Z y) {return x += y;}
    friend Z operator - (Z x, Z y) {return x -= y;}
    friend Z operator * (Z x, Z y) {return x *= y;}
    friend Z operator / (Z x, Z y) {return x /= y;}
    friend Z operator ^ (Z x, Z y) {return x ^= y;}
    friend Z operator % (Z x, Z y) {return x %= y;}
    friend bool operator == (Z x, Z y) {return x.x == y.x;}
    friend bool operator != (Z x, Z y) {return x.x != y.x;}
    friend istream& operator >> (istream &is, Z& o) {T val; is >> val; o
      = Z(val); return is;}
    friend ostream& operator << (ostream &os, const Z &z) {return os <<
      z.x;}
};
```

3.3 Fraction

```
template <typename T>
class Frac
{
    T p, q; // p / q
    void reduce()
    {
        assert(q != 0);
        T g = gcd(p, q);
        p /= g, q /= g;
        if (q < 0) p = -p, q = -q;
    }
    public :
    Frac(T p = 0, T q = 1) : p(p), q(q) {assert(q != 0); reduce();}
    T get_p() {return p;}
    T get_q() {return q;}
    double get_val() {return 1.0 * p / q;}
    Frac operator + (const Frac &o) {return Frac(p * o.q + o.p * q, q *
      o.q);}
    Frac operator - (const Frac &o) {return Frac(p * o.q - o.p * q, q *
      o.q);}
    Frac operator * (const Frac &o) {return Frac(p * o.p, q * o.q);}
    Frac operator / (const Frac &o) {return Frac(p * o.q, q * o.p);}
    Frac& operator += (const Frac &o) {*this = *this + o; return *this;}
    Frac& operator -= (const Frac &o) {*this = *this - o; return *this;}
    Frac& operator *= (const Frac &o) {*this = *this * o; return *this;}
    Frac& operator /= (const Frac &o) {*this = *this / o; return *this;}
    Frac operator -() {return Frac(-p, q);}
    bool operator >= (const Frac &o) {return p * o.q >= o.p * q;}
    bool operator > (const Frac &o) {return p * o.q > o.p * q;}
    bool operator == (const Frac &o) {return p * o.q == o.p * q;}
    bool operator <= (const Frac &o) {return !(*this > o);}
    bool operator < (const Frac &o) {return !(*this >= o);}
    bool operator != (const Frac &o) {return !(*this == o);}
};
```

3.4 Combination

```
template <typename T, int MOD = 1000000007>
class COM
{
    int n;
    vector <Z <T, MOD>> fac, inv;
    public :
    COM(int n) : n(n), fac(2 * n + 2), inv(2 * n + 2)
    {
        fac[0] = inv[0] = inv[1] = 1;
        for (int i = 1; i <= 2 * n; ++i) fac[i] = fac[i - 1] * i;
        inv[2 * n] = fac[2 * n] ^ (MOD - 2);
        for (int i = 2 * n - 1; i > 0; --i) inv[i] = inv[i + 1] * (i + 1)
      ;
    }
    Z <T, MOD> f(int x) {return fac[x];}
    Z <T, MOD> inv_f(int x) {return inv[x];}
    Z <T, MOD> comb(int x, int y)
    {
        if (x < 0 || y < 0 || y > x) return 0;
        else return fac[x] * inv[x - y] * inv[y];
    }
    Z <T, MOD> arr(int x, int y)
    {
        if (x < 0 || y < 0 || y > x) return 0;
        else return fac[x] * inv[x - y];
    }
    Z <T, MOD> catalan(int x) {return comb(2 * x, x) / (x + 1);}
};
```

3.5 The Sieve of Primes & Euler's Totient Function & MÃűbius Function

```
class Prime
{
    int mx;
    vector <int> p, phi, mu;
    void pre()
    {
        vector <int> fl(mx + 1, 0);
        phi[1] = mu[1] = 1;
        for (int i = 2; i <= mx; ++i)
        {
            if (!fl[i]) p.push_back(i), phi[i] = i - 1, mu[i] = -1;
            for (auto v : p)
            {
                if (i * v > mx) break;
                fl[i * v] = 1;
                if (i % v == 0) {phi[i * v] = phi[i] * v, mu[i * v] = 0;
      break;}
                phi[i * v] = phi[i] * (v - 1); mu[i * v] = -mu[i];
            }
        }
    }
    public :
    Prime(int mx) : mx(mx), phi(mx + 1, 0), mu(mx + 1, 0) {pre();}
    auto get_prime() {return p;}
    auto get_phi() {return phi;}
    auto get_mu() {return mu;}
};
```

3.6 Exgcd

```
ll exgcd(ll a, ll b, ll &x, ll &y)
{
    if (!b) {x = 1; y = 0; return abs(a);}
    ll d = exgcd(b, (a % b + b) % b, y, x);
    y -= a / b * x;
    return d;
}
```

3.7 CRT

```
ll CRT()
{
    //ans % a[i] = b[i] if gcd (a[i],a[j]) = 1
    ll sum = 1, ans = 0;
    for (int i = 1; i <= n; ++i) sum *= a[i];
    for (int i = 1; i <= n; ++i)
    {
        ll x, y, tmp = sum / a[i];
        exgcd(tmp, a[i], x, y);
        ans = (ans + tmp * x * b[i]) % sum;
    }
```

```
    return (ans + sum) % sum;
}
```

3.8 Lagrange Interpolation

```
ll Lagrange(ll num)
{
    ll ans = 0;
    for (int i = 1; i <= n; ++i)
    {
        ll p = y[i] % MOD, q = 1;
        for (int j = 1; j <= n; ++j)
        {
            if (i == j) continue;
            p = p * (num - x[j]) % MOD;
            q = q * (x[i] - x[j]) % MOD;
        }
        ans = (ans + p * qpow(q, MOD - 2) % MOD + MOD) % MOD;
    }
    return ans;
}
```

3.9 Max-Plus Convolution

```
inline vector<ll> mul(const vector<ll> &f,const vector<ll> &g)
{
    int fn=int(f.size())-1,gn=int(g.size())-1;
    int qn=fn+gn;
    vector<ll> Q(qn+1,-INF);
    int sf=0,tf=fn,sg=0,tg=gn;
    while(sf<=fn&&f[sf]==-INF)++sf;
    while(tf>=sf&&f[tf]==-INF)--tf;
    while(sg<=gn&&g[sg]==-INF)++sg;
    while(tg>=sg&&g[tg]==-INF)--tg;
    if(sf>tf||sg>tg)return Q;
    int i=sf,j=sg;
    if(i+j>qn)return Q;
    Q[i+j]=f[i]+g[j];
    while(i+j<qn&&(i<tf||j<tg))
    {
        if(j==tg||(i<tf&&f[i+1]-f[i]>=g[j+1]-g[j]))++i;
        else ++j;
        Q[i+j]=f[i]+g[j];
    }
    return Q;
}
```

3.10 Gaussian Elimination

```
vector <vector <double>> a(n + 1, vector <double> (n + 2)); // n * (n + 1)
for (int i = 1; i <= n; ++i)
{
    int p = cur, ok = 0;
    while (p <= n)
    {
        if (a[p][i] != 0) {ok = 1; break;}
        ++p;
    }
    if (!ok) continue;
    for (int j = i; j <= n + 1; ++j) swap(a[p][j], a[cur][j]);
    for (int j = n + 1; j >= i; --j) a[cur][j] /= a[cur][i];
    for (int j = 1; j <= n; ++j)
    {
        if (j == cur) continue;
        for (int k = n + 1; k >= i; --k) a[j][k] -= a[cur][k] * a[j][i];
    }
    ++cur;
}
if (cur <= n) puts("No Solution"); //0 solution or infinte solutions
else {for (int i = 1; i <= n; ++i) printf("%.6lf\n", a[i][n + 1]);}
```

3.11 FFT

```
using cd = complex <double>;
class FFT
{
    const double Pi = acos(-1);
    vector <int> rev; int n = 1;
    void fft(vector <cd> &a, int op)
    {
        for (int i = 0; i < n; ++i)
            if (i < rev[i]) swap(a[i], a[rev[i]]);
        for (int len = 1; len < n; len <<= 1)
        {
            cd wn(cos(Pi / len), op * sin(Pi / len));
            for (int i = 0; i < n; i += len << 1)
            {
                cd w(1);
                for(int j = 0; j < len; ++j)
                {
                    cd u = a[i + j], v = a[i + j + len] * w;
                    a[i + j] = u + v; a[i + j + len] = u - v;
                    w *= wn;
                }
            }
        }
        if (op == -1) {for (auto &x : a) x /= n;}
    }
    public :
    vector <double> conv(vector <cd> a, vector <cd> b)
    {
        int L = 0, tot = (int)a.size() + (int)b.size() - 1; n = 1;
        while (n < tot) n <<= 1, ++L;
        rev.resize(n); a.resize(n); b.resize(n);
        for (int i = 0; i < n; ++i) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (L - 1));
        fft(a, 1); fft(b, 1);
        for (int i = 0; i < n; ++i) a[i] = a[i] * b[i];
        fft(a, -1);
        vector <double> ans(tot);
        for (int i = 0; i < tot; ++i) ans[i] = a[i].real();
        return ans;
    }
};
```

3.12 NTT

```
template <int MOD = 998244353>
class NTT
{
    vector <int> rev; int n = 1;
    const int P = MOD; //998244353 469762049 5000000001507329LL 4179340454199820289
    const int G = 3;
    //1945555039024054273 for G = 5
    int qpow(int x, int y)
    {
        int res = 1;
        while (y)
        {
            if (y & 1) res = 1ll * res * x % P;
            x = 1ll * x * x % P;
            y >>= 1;
        }
        return res;
    }
    void init_rev(int tot)
    {
        int L = 0; n = 1;
        while (n < tot) n <<= 1, ++L;
        rev.resize(n);
        for (int i = 0; i < n; ++i) rev[i] = (rev[i >> 1] >> 1) | ((i & 1) << (L - 1));
    }
    void ntt(vector <int> &a, int op)
    {
        for (int i = 0; i < n; ++i)
            if (i < rev[i]) swap(a[i], a[rev[i]]);
        for (int len = 1; len < n; len <<= 1)
        {
            int wn = qpow(G, (P - 1) / (len << 1));
            if (op == -1) wn = qpow(wn, P - 2);
            for (int i = 0; i < n; i += len << 1)
            {
                int w = 1;
                for(int j = 0; j < len; ++j)
                {
                    int u = a[i + j] >= P ? a[i + j] - P : a[i + j], v = 1ll * a[i + j + len] * w % P;
                    a[i + j] = u + v >= P ? u + v - P : u + v;
                    a[i + j + len] = u - v <= 0 ? u - v + P : u - v;
                    w = 1ll * w * wn % P;
                }
            }
        }
        if (op == -1)
        {
            int inv_n = qpow(n, P - 2);
            for (auto &x : a) x = 1ll * x * inv_n % P;
        }
    }
    public:
    vector <int> Conv(vector <int> a, vector <int> b)
    {
        int tot = (int)a.size() + (int)b.size() - 1;
        init_rev(tot);
```

```
        a.resize(n); b.resize(n);
        ntt(a, 1); ntt(b, 1);
        for (int i = 0; i < n; ++i) a[i] = 1ll * a[i] * b[i] % P;
        ntt(a, -1); a.resize(tot);
        return a;
    }
    vector <int> Inv(vector <int> a, int m)
    {
        int inv = qpow(a[0], P - 2);
        vector <int> b{inv};
        for (int len = 2; (len >> 1) < m; len <<= 1)
        {
            int sz = min((int)a.size(), len);
            vector <int> f(a.begin(), a.begin() + sz);
            init_rev(len << 1);
            f.resize(n);
            vector <int> h = b; h.resize(n);
            ntt(f, 1); ntt(h, 1);
            for (int i = 0; i < n; ++i)
                h[i] = 1ll * h[i] * (2ll - 1ll * f[i] * h[i] % P + P) %
      P;
            ntt(h, -1);
            b.assign(h.begin(), h.begin() + min(len, m));
        }
        b.resize(m);
        return b;
    }
    pair <vector <int>, vector <int>> Divmod(vector <int> a, vector <int>
       b)
    {
        int n_sz = a.size(), m_sz = b.size();
        if (n_sz < m_sz) return {{0}, a};
        int dq = n_sz - m_sz + 1;
        vector <int> ra = a, rb = b;
        reverse(ra.begin(), ra.end()); reverse(rb.begin(), rb.end());
        ra.resize(dq); rb.resize(dq);
        vector <int> inv_rb = Inv(rb, dq);
        vector <int> q = Conv(ra, inv_rb);
        q.resize(dq);
        reverse(q.begin(), q.end());
        vector <int> qb = Conv(q, b);
        vector <int> r(m_sz - 1);
        for (int i = 0; i < m_sz - 1; ++i)
            r[i] = (0ll + a[i] - (i < (int)qb.size() ? qb[i] : 0) + P) %
       P;
        return {q, r};
    }
    vector <int> Sqrt(vector <int> a, int m) // b[0] = 1
    {
        vector <int> b{1};
        int inv2 = qpow(2, P - 2);
        for (int len = 2; (len >> 1) < m; len <<= 1)
        {
            int sz = min((int)a.size(), len);
            vector <int> f(a.begin(), a.begin() + sz);
            vector <int> inv_b = Inv(b, len);
            init_rev(len << 1);
            f.resize(n); inv_b.resize(n);
            ntt(f, 1); ntt(inv_b, 1);
            for (int i = 0; i < n; ++i)
                f[i] = 1ll * f[i] * inv_b[i] % P;
            ntt(f, -1);
            b.resize(len);
            for (int i = 0; i < len; ++i)
                b[i] = 1ll * (b[i] + f[i]) % P * inv2 % P;
        }
        b.resize(m);
        return b;
    }
};
```

3.13 FWT

```
template <typename T>
class FWT
{
    public:
    const int P = 998244353;
    const ll inv_2 = 499122177; //500000004 for 1e9 + 7
    private:
    void Or(vector <T> &F, int op = 1)
    {
        int n = F.size();
        for (int i = 2; i <= n; i <<= 1)
        {
            int o = i >> 1;
            for (int j = 0; j < n; j += i)
                for (int k = 0; k < o; ++k) F[j + k + o] = (F[j + k + o]
      + F[j + k] * op % P) % P;
        }
    }
    void And(vector <T> &F, int op = 1)
    {
        int n = F.size();
        for (int i = 2; i <= n; i <<= 1)
        {
            int o = i >> 1;
            for (int j = 0; j < n; j += i)
                for (int k = 0; k < o; ++k) F[j + k] = (F[j + k] + F[j +
      k + o] * op % P) % P;
        }
    }
    void Xor(vector <T> &F, int op = 1)
    {
        int n = F.size();
        for (int i = 2; i <= n; i <<= 1)
        {
            int o = i >> 1;
            for (int j = 0; j < n; j += i)
            {
                for (int k = 0; k < o; ++k)
                {
                    F[j + k] = (F[j + k] + F[j + k + o]) % P;
                    F[j + k + o] = (F[j + k] - 2 * F[j + k + o] % P + P)
       % P;
                    F[j + k] = F[j + k] * op % P;
                    F[j + k + o] = F[j + k + o] * op % P;
                }
            }
        }
    }
    vector <T> mul(vector <T> A, vector <T> B)
    {
        int n = A.size();
        vector <T> C(n);
        for (int i = 0; i < n; ++i) C[i] = A[i] * B[i] % P;
        return C;
    }
    public :
    FWT() {}
    vector <T> calc_Or(vector <T> A, vector <T> B)
    {
        Or(A); Or(B);
        auto C = mul(A, B);
        Or(C, P - 1);
        return C;
    }
    vector <T> calc_And(vector <T> A, vector <T> B)
    {
        And(A); And(B);
        auto C = mul(A, B);
        And(C, P - 1);
        return C;
    }
    vector <T> calc_Xor(vector <T> A, vector <T> B)
    {
        Xor(A); Xor(B);
        auto C = mul(A, B);
        Xor(C, inv_2);
        return C;
    }
};
```

4 Computational Geometry

```cpp
using LD = long double;
const LD pi = acos(-1.0);
const LD eps = 1e-8L;
int dcmp(LD x) {return x < -eps ? -1 : (x > eps ? 1 : 0);}
struct Point {LD x, y; Point(LD x = 0, LD y = 0) : x(x), y(y) {}};
struct Circle {Point O; LD r; Circle(Point O = Point(), LD r = 0) : O(O), r(r) {}};
typedef Point Vector;
Vector operator + (Vector A, Vector B) {return Vector(A.x + B.x, A.y + B.y);}
Vector operator - (Vector A, Vector B) {return Vector(A.x - B.x, A.y - B.y);}
Vector operator * (Vector A, LD k) {return Vector(A.x * k, A.y * k);}
Vector operator / (Vector A, LD k) {return Vector(A.x / k, A.y / k);}
bool operator == (Vector A, Vector B) {return dcmp(A.x - B.x) == 0 && dcmp(A.y - B.y) == 0;}
bool operator != (Vector A, Vector B) {return !(A == B);}

LD dot(Vector A, Vector B) {return A.x * B.x + A.y * B.y;}
LD dis(Point A, Point B) {return sqrt((A.x - B.x) * (A.x - B.x) + (A.y - B.y) * (A.y - B.y));}
LD cross(Vector A, Vector B) {return A.x * B.y - A.y * B.x;} // A -> B counter-clockwise if cross (A,B) > 0
LD len(Point A)  {return sqrt(A.x * A.x + A.y * A.y);}
LD angle(Vector A, Vector B) {return acos(dot(A, B) / (len(A) * len(B)));}
Vector proj(Vector A, Vector B) {return A * (dot(A, B) / dot(A, A));} // project onto A
Point foot(Point P, Point A, Point B) {Vector AP = P - A, AB = B - A; return A + proj(AB, AP);} // foot
Point reflect(Point P, Point A, Point B) {Point F = foot(P, A, B); return F * 2 - P;} // symmetry point
Point rotate(Point P, LD theta) {return (Point){P.x * cos(theta) - P.y * sin(theta), P.x * sin(theta) + P.y * cos(theta)};} // counterclockwise
bool chk_on_line(Point P, Point A, Point B) {return dcmp(cross(P - A, B - A)) == 0;} // judge whether on line AB
bool chk_on_seg(Point P, Point A, Point B) {return chk_on_line(P, A, B) && dcmp(dot(P - A, P - B)) <= 0;} // judge whether on segment AB
pair <Point, Point> get_parallel(Point P, Point A, Point B) {return {P, P + (B - A)};}// the line passing through P and parallel to line AB
pair <Point, Point> get_vertical(Point P, Point A, Point B) // the line passing through P and perpendicular to line AB
{
    Vector V = B - A, N = {-V.y, V.x};
    return {P, P + N};
}
LD dis_seg(Point P, Point A, Point B) // the distance from point P to segment AB.
{
    if (dcmp(dot(B - A, P - A)) < 0) return dis(P, A);
    if (dcmp(dot(A - B, P - B)) < 0) return dis(P, B);
    return fabs(cross(P - A, P - B)) / dis(A, B);
}
Point ll_inter_pt(Point A, Point B, Point C, Point D) {return A + (B - A) * cross(C - A, D - C) / cross(B - A, D - C);}
bool chk_ll_inter(Point A, Point B, Point C, Point D) {return dcmp(cross(B - A, D - C)) != 0;} // line - line
bool chk_ls_inter(Point A, Point B, Point C, Point D) {return chk_on_seg(ll_inter_pt(A, B, C, D), C, D);} // The intersection of AB(line) and CD (line) is on the CD (seg).
bool chk_ss_inter(Point A, Point B, Point C, Point D) // seg - seg
{
    LD c1 = cross(B - A, C - A), c2 = cross(B - A, D - A);
    LD d1 = cross(D - C, A - C), d2 = cross(D - C, B - C);
    if (dcmp(c1) * dcmp(c2) < 0 && dcmp(d1) * dcmp(d2) < 0) return true;
    if (dcmp(c1) == 0 && chk_on_seg(C, A, B)) return true;
    if (dcmp(c2) == 0 && chk_on_seg(D, A, B)) return true;
    if (dcmp(d1) == 0 && chk_on_seg(A, C, D)) return true;
    if (dcmp(d2) == 0 && chk_on_seg(B, C, D)) return true;
    return false;
}
bool SameSide(Point A, Point B, Point C, Point D) {return cross(A - C, D - C) * cross(D - C, B - C) < 0;} // Point A,B  Line C,D

LD area(vector <Point> &P)
{
    int n = P.size();
    LD res = 0;
    for (int i = 0; i < n; ++i) res += cross(P[i], P[(i + 1) % n]);
    return res / 2.0;
}
bool is_convex(vector <Point> &P)
{
    int n = P.size();
    for(int i = 0; i < n - 1; ++i)
        if (dcmp(cross(P[i + 1] - P[i], P[(i + 2) % n] - P[i])) < 0)
      return false;
    return true;
}
int in_Poly(vector <Point> &P, Point A) // O (n) for any polygons
{
    int cnt = 0, n = P.size();
    for (int i = 0; i < n; ++i)
    {
        int j = (i + 1) % n;
        if (chk_on_seg(A, P[i], P[j])) return 2; // on the edge
        if (A.y >= min(P[i].y, P[j].y) && A.y < max(P[i].y, P[j].y)) // the intersection is on the right
            cnt += dcmp(((A.y - P[i].y) * (P[j].x - P[i].x) / (P[j].y - P[i].y) + P[i].x) - A.x) > 0;
    }
    return cnt & 1;
}
int in_convex_Poly(vector <Point> &P, Point A) // O (log n) only for convex polygons
{
    int n = P.size();
    if (chk_on_seg(A, P[0], P[1]) || chk_on_seg(A, P[0], P[n - 1]))
      return 2;
    if (dcmp(cross(P[n - 1] - P[0], A - P[0])) > 0 || dcmp(cross(P[1] - P[0], A - P[0])) < 0) return 0;
    int l = 1, r = n - 2, res = -1;
    while (l <= r)
    {
        int mid = (l + r) >> 1;
        if (dcmp(cross(P[mid] - P[0], A - P[0])) >= 0) res = mid, l = mid + 1;
        else r = mid - 1;
    }
    if (chk_on_seg(A, P[res], P[res + 1])) return 2;
    if (dcmp(cross(P[res + 1] - P[res], A - P[res])) < 0) return 0;
    return 1;
}
auto convex_hull(vector <Point> &P) // strict convex hull (<= 0)
{
    int n = P.size();
    sort(P.begin(), P.end(), [] (Point &x, Point &y) {return x.x == y.x ? x.y < y.y : x.x < y.x;});
    vector <Point> hull;
    hull.resize(2 * n + 1);
    int k = 0;
    for (int i = 0; i < n; ++i)
    {
        while (k >= 2 && dcmp(cross(hull[k - 1] - hull[k - 2], P[i] - hull[k - 2])) <= 0) --k;
        hull[k++] = P[i];
    }
    for (int i = n - 2, t = k; i >= 0; --i)
    {
        while (k > t && dcmp(cross(hull[k - 1] - hull[k - 2], P[i] - hull[k - 2])) <= 0) --k;
        hull[k++] = P[i];
    }
    hull.resize(k - 1);
    return hull;
}
LD diameter(vector <Point> &P)
{
    int n = P.size();
    if (n <= 1) return 0;
    if (n == 2) return len(P[1] - P[0]);
    LD res = 0;
    for (int i = 0, j = 2; i < n; ++i)
    {
        while (dcmp(cross(P[(i + 1) % n] - P[i], P[j] - P[i]) - cross(P[(i + 1) % n] - P[i], P[(j + 1) % n] - P[i])) <= 0) j = (j + 1) % n;
        res = max(res, max(len(P[j] - P[i]), len(P[j] - P[(i + 1) % n])));
    }
    return res;
}
template <typename Line = pair <Point, Point>>
LD half_plane(vector <Line> &Vec)
{
    int n = Vec.size();
    auto get_angle = [&] (Line line) -> LD {return atan2(line.second.y - line.first.y, line.second.x - line.first.x);};
    sort(Vec.begin(), Vec.end(), [&] (Line A, Line B)
    {
        LD angA = get_angle(A), angB = get_angle(B);
        return fabs(angA - angB) > eps ? angA < angB : cross(A.second - A.first, B.second - A.first) < 0;
    });
    int h = 1, t = 0;
    auto check = [&] (Line x, Line y, Line z) -> bool
    {
        Point P = ll_inter_pt(y.first, y.second, z.first, z.second);
        return dcmp(cross(x.second - x.first, P - x.first)) < 0;
    };
    vector <Line> q(n + 10); q[++t] = Vec[0];
    for (int i = 1; i < n; ++i)
    {
        if (get_angle(Vec[i]) - get_angle(Vec[i - 1]) < eps) continue;
        while (h < t && check(Vec[i], q[t], q[t - 1])) --t;
        while (h < t && check(Vec[i], q[h], q[h + 1])) ++h;
        q[++t] = Vec[i];
```

```
    }
    while (h < t && check(q[h], q[t], q[t - 1])) --t;
    q[++t] = q[h];
    vector <Point> p;
    for (int i = h; i < t; ++i) p.push_back(ll_inter_pt(q[i].first, q[i].second, q[i + 1].first, q[i + 1].second));
    return area(p);
}
vector <Point> minkowski(vector <Point> &vecA, vector <Point> &vecB)
{
    int n = vecA.size(), m = vecB.size();
    vector <Point> A(n), B(m), C {vecA[0] + vecB[0]};
    for (int i = 0; i < n; ++i) A[i] = vecA[(i + 1) % n] - vecA[i];
    for (int i = 0; i < m; ++i) B[i] = vecB[(i + 1) % m] - vecB[i];
    int posa = 0, posb = 0;
    while (posa < n || posb < m)
    {
        if (posa == n) C.push_back(C.back() + B[posb++]);
        else if (posb == m) C.push_back(C.back() + A[posa++]);
        else if (dcmp(cross(A[posa], B[posb])) >= 0) C.push_back(C.back() + A[posa++]);
        else C.push_back(C.back() + B[posb++]);
    }
    return convex_hull(C);
}
bool chk_convex_inter_strict(vector <Point> A, vector <Point> B) // Check if two convex polygons strictly intersect
{
    vector <Point> negB;
    for (auto p : B) negB.push_back(Point(-p.x, -p.y));
    negB = convex_hull(negB);
    vector <Point> C = minkowski(A, negB);
    if (C.size() < 3) return false;
    Point origin(0, 0);
    return in_convex_Poly(C, origin) == 1;
}

bool chk_in_cir(Circle C, Point P) {return dcmp(len(P - C.O) - C.r) <= 0;}
Point get_cir_pt(Circle C, LD theta) {return {C.O.x + C.r * cos(theta), C.O.y + C.r * sin(theta)};}
int chk_lc_inter(Point A, Point B, Circle C)
{
    LD d = dis_seg(C.O, A, B);
    if (dcmp(d - C.r) == 0) return 0; // tangent
    if (dcmp(d - C.r) > 0) return -1; // separation
    return 1; // intersection
}
int chk_cc_inter(Circle A, Circle B) // the number of tagent lines
{
    LD d = len(A.O - B.O);
    if (dcmp(A.r + B.r - d) < 0) return 4; // externally separate
    if (dcmp(A.r + B.r - d) == 0) return 3; // externally tangent
    if (dcmp(fabs(A.r - B.r) - d) == 0) return 1; // internally tangent
    if (dcmp(fabs(A.r - B.r) - d) > 0) return 0; // one circle inside the other
    return 2; // intersection
}
pair <Point, Point> lc_inter(Point A, Point B, Circle C)
{
    Point F = foot(C.O, A, B); LD d = dis(C.O, F);
    Vector E = (B - A) / dis(A, B);
    Point P1 = F - E * sqrt(C.r * C.r - d * d);
    Point P2 = F + E * sqrt(C.r * C.r - d * d);
    return {P1, P2};
}
pair <Point, Point> cc_inter(Circle A, Circle B)
{
    Vector k = B.O - A.O;
    LD d = len(k);
    LD alpha = atan2(k.y, k.x), beta = acos((A.r * A.r + d * d - B.r * B.r) / (2 * A.r * d));
    Point P1 = get_cir_pt(A, alpha - beta), P2 = get_cir_pt(A, alpha + beta);
    return  {P1, P2};
}
pair <Point, Point> tan_cir(Point P, Circle C)
{
    LD d = len(C.O - P), theta = asin(C.r / d);
    Vector E = (C.O - P) / d;
    Vector P1 = P + (rotate(E, theta) * sqrt(d * d - C.r * C.r));
    Vector P2 = P + (rotate(E, -theta) * sqrt(d * d - C.r * C.r));
    return {P1, P2};
}
Circle tri_incir(Point A, Point B, Point C)
{
    LD a = dis(B, C), b = dis(A, C), c = dis(A, B);
    Point O = (A * a + B * b + C * c) / (a + b + c);
    return {O, dis_seg(O, A, B)};
}
Circle tri_circum(Point A, Point B, Point C)
{
    LD Bx = B.x - A.x, By = B.y - A.y, Cx = C.x - A.x, Cy = C.y - A.y;
    LD D = 2 * (Bx * Cy - By * Cx);
    LD x = (Cy * (Bx * Bx + By * By) - By * (Cx * Cx + Cy * Cy)) / D + A.x;
    LD y = (Bx * (Cx * Cx + Cy * Cy) - Cx * (Bx * Bx + By * By)) / D + A.y;
    Point P(x, y);
    return Circle(P, dis(A, P));
}
auto get_tangents(Circle A, Circle B)
{
    vector <pair <Point, Point>> tangents;
    LD d = len(A.O - B.O), dif = A.r - B.r, sum = A.r + B.r;
    if (dcmp(d - fabs(dif)) < 0) return tangents;
    LD base = atan2(B.O.y - A.O.y, B.O.x - A.O.x);
    if (dcmp(d - fabs(dif)) == 0)
    {
        tangents.push_back({get_cir_pt(A, base + (A.r < B.r ? pi : 0)), get_cir_pt(A, base + (A.r < B.r ? pi : 0))});
        return tangents;
    }
    LD theta = acos(dif / d);
    tangents.push_back({get_cir_pt(A, base + theta), get_cir_pt(B, base + theta)});
    tangents.push_back({get_cir_pt(A, base - theta), get_cir_pt(B, base - theta)});
    if (dcmp(d - sum) == 0) tangents.push_back({get_cir_pt(A, base), get_cir_pt(A, base)});
    if (dcmp(d - sum) > 0)
    {
        theta = acos(sum / d);
        tangents.push_back({get_cir_pt(A, base + theta), get_cir_pt(B, base + theta + pi)});
        tangents.push_back({get_cir_pt(A, base - theta), get_cir_pt(B, base - theta + pi)});
    }
    return tangents;
}
LD tri_cir_area(Point A, Point B, Circle C)
{
    Vector OA = A - C.O, OB = B - C.O;
    LD S = cross(OA, OB), sign = dcmp(cross(OA, OB)) > 0 ? 1 : -1;
    bool da = dcmp(len(OA) - C.r) < 0, db = dcmp(len(OB) - C.r) < 0;
    if (dcmp(S) == 0) return 0;
    if (da && db) return S * 0.5; // triangle
    if (!da && !db)
    {
        if (chk_lc_inter(A, B, C) == 1)// arc + triangle + arc
        {
            auto [P1, P2] = lc_inter(A, B, C);
            Vector OP1 = P1 - C.O, OP2 = P2 - C.O;
            if (dis(A, P1) > dis(A, P2)) swap(P1, P2);
            return cross(OP1, OP2) * 0.5 + sign * 0.5 * C.r * C.r * (angle(OA, OP1) + angle(OB, OP2));
        }
        else return sign * 0.5 * C.r * C.r * angle(OA, OB); // arc
    }
    else // triangle + arc
    {
        auto [P1, P2] = lc_inter(A, B, C);
        if (chk_on_seg(P2, A, B)) swap(P1, P2);
        Vector OP1 = P1 - C.O;
        if (dcmp(len(OA) - C.r) < 0) return cross(OA, OP1) * 0.5 + sign * 0.5 * C.r * C.r * angle(OP1, OB);
        else return cross(OP1, OB) * 0.5 + sign * 0.5 * C.r * C.r * angle(OP1, OA);
    }
}
LD cc_area(Circle C1, Circle C2)
{
    int op = chk_cc_inter(C1, C2);
    if (op <= 1) return pi * min(C1.r, C2.r) * min(C1.r, C2.r);
    else if (op == 4) return 0;
    else
    {
        LD d = dis(C1.O, C2.O);
        LD alpha = 2 * acos((C1.r * C1.r - C2.r * C2.r + d * d) / (2 * C1.r * d));
        LD beta = 2 * acos((C2.r * C2.r - C1.r * C1.r + d * d) / (2 * C2.r * d));
        return 0.5 * (C1.r * C1.r * (alpha - sin(alpha)) + C2.r * C2.r * (beta - sin(beta)));
    }
}
```

5 String

5.1 Hash

```cpp
template <unsigned long long base = 13331>
class Hash
{
    using u64 = unsigned long long;
    vector <u64> pw, hsh;
    public:
    Hash(char *s)
    {
        int n = strlen(s);
        pw.assign(n + 1, 0), hsh.assign(n + 1, 0);
        pw[0] = 1; hsh[0] = 0;
        for (int i = 1; i <= n; ++i) pw[i] = pw[i - 1] * base, hsh[i] = hsh[i - 1] * base + s[i - 1];
    }
    u64 get(int l, int r) {return hsh[r] - hsh[l - 1] * pw[r - l + 1];}
    u64 link(int l1, int r1, int l2, int r2) {return get(l1, r1) * pw[r2 - l2 + 1] + get(l2, r2);}
    bool same(int l1, int r1, int l2, int r2) {return get(l1, r1) == get(l2, r2);}
};
```

5.2 KMP

```cpp
class KMP
{
    vector <int> fail, ans;
    void getfail(char *s) // Match a with b
    {
        int len = strlen(s);
        fail.assign(len + 1, 0);
        fail[0] = -1;
        for (int i = 1; i < len; ++i)
        {
            int cnt = fail[i - 1];
            while (cnt >= 0 && s[cnt + 1] != s[i]) cnt = fail[cnt];
            if (s[cnt + 1] == s[i]) ++cnt;
            fail[i] = cnt;
        }
    }
    public:
    auto get_pos(char *s, char *t)
    {
        getfail(t);
        int lens = strlen(s), lent = strlen(t), cnt = -1;
        for (int i = 0; i < lens; ++i)
        {
            while (cnt >= 0 && t[cnt + 1] != s[i]) cnt = fail[cnt];
            if (t[cnt + 1] == s[i])
            {
                ++cnt;
                if (cnt + 1 == lent) ans.push_back(i - lent + 2), cnt = fail[cnt];
            }
        }
        return ans;
    }
};
```

5.3 Manacher

```cpp
int Manacher(char *s)
{
    int n = strlen(s), cnt = 0, r = 0, mid = 0, ans = 0;
    vector <char> a(2 * n + 5); vector <int> p(2 * n + 5);
    a[++cnt] = '!'; a[++cnt] = '#';
    for (int i = 0; i < n; ++i) a[++cnt] = s[i], a[++cnt] = '#';
    a[++cnt] = '~';
    for (int i = 2; i < cnt; ++i)
    {
        if (i <= r) p[i] = min(r - i + 1, p[mid * 2 - i]);
        else p[i] = 1;
        while (a[i - p[i]] == a[i + p[i]]) ++p[i];
        if (i + p[i] > r) r = i + p[i] - 1, mid = i;
        ans = max(ans, p[i]);
    }
    return ans - 1;
}
```

5.4 Trie

5.4.1 Trie

```cpp
struct Trie
{
    int n, m, cnt; //m total len
    vector <vector <int>> ch; vector <int> vis;
    public:
    Trie(int n, int m) : n(n), m(m), ch(m, vector <int> (26, 0)), vis(m){cnt = 0;}
    void insert(char *s)
    {
        int u = 0, len = strlen(s);
        for (int i = 0; i < len; ++i)
        {
            int c = s[i] - 'a';
            if (!ch[u][c]) ch[u][c] = ++cnt;
            u = ch[u][c];
        }
        ++vis[u];
    }
    int query(char *s)
    {
        int u = 0, len = strlen(s);
        for (int i = 0; i < len; ++i)
        {
            int c = s[i] - 'a';
            if (!ch[u][c]) return 0;
            u = ch[u][c];
        }
        return vis[u];
    }
};
```

5.4.2 01-Trie

```cpp
class Trie
{
    int n, cnt;
    vector <vector <int>> ch;
    vector <int> val, w;
    public :
    Trie(int n, int lg) : n(n), val(2 * lg * n + 1, 0), w(2 * lg * n + 1, 0), ch(2 * lg * n + 1, vector <int> (2, 0)) {cnt = 1;}
    void pushup(int u)
    {
        w[u] = val[u] = 0;
        //w[u] Number of values (weights) on the edge between node u and its parent node
        //val[u] XOR sum maintained by the subtree rooted at u
        if (ch[u][0]) w[u] ^= w[ch[u][0]], val[u] ^= val[ch[u][0]] << 1;
        if (ch[u][1]) w[u] ^= w[ch[u][1]], val[u] ^= (val[ch[u][1]] << 1) | w[ch[u][1]];
    }
    void modify(int &u, int v, int dep)
    {
        if (!u) u = ++cnt;
        w[u] ^= 1;
        if (dep < 0) return ;
        modify(ch[u][v & 1], v >> 1, dep - 1);
        pushup(u);
    }
    void erase(int u, int v, int dep)
    {
        if (!u) return ;
        w[u] ^= 1;
        if (dep < 0) return ;
        erase(ch[u][v & 1], v >> 1, dep - 1);
        pushup(u);
    }
    void add(int u) // add 1 in [1,n]
    {
        swap(ch[u][0], ch[u][1]);
        if (ch[u][0]) add(ch[u][0]);
        pushup(u);
    }
};
```

5.5 Aho-Corasick Automaton

```cpp
struct ACAM
{
    struct Node
    {
        int ch[27],tot,nxt;bool vis;
        Node(){memset(ch,0,sizeof(ch));tot=nxt=0;vis=false;}
    };
    vector<Node> a;queue<int> q;
    inline void Init(){a.assign(1,Node());return;}
    ACAM(){a.assign(1,Node());return;}
    inline int newNode()
    {
        int tmp=a.size();
```

```
        a.push_back(Node());
        return tmp;
    }
    inline void ins(char *s)
    {
        int p=0,len=strlen(s);
        rep(0,len-1,i)
        {
            int x=a[p].ch[s[i]-'a'];
            if(!x)x=newNode(),a[p].ch[s[i]-'a']=x;p=x;
        }
        a[p].tot++;

    }
    inline void build()
    {
        while(!q.empty())q.pop();
        rep(0,26,i)if(a[0].ch[i])
        {
            q.push(a[0].ch[i]);
            a[a[0].ch[i]].nxt=0;
        }
        while(!q.empty())
        {
            int u=q.front();q.pop();
            rep(0,26,i)
            {
                if(a[u].ch[i])
                {
                    q.push(a[u].ch[i]);
                    a[a[u].ch[i]].nxt=a[a[u].nxt].ch[i];
                }
                else
                {
                    a[u].ch[i]=a[a[u].nxt].ch[i];
                }
            }
        }
    }
    inline int qry(char *s)
    {
        for(Node&v:a)v.vis=false;
        int len=strlen(s),p=0,ans=0;
        rep(0,len-1,i)
        {
            p=a[p].ch[s[i]-'a'];
            for(int j=p;!a[j].vis&&j;j=a[j].nxt)
            {
                ans+=a[j].tot;
                a[j].vis=true;
            }
        }
        return ans;
    }
};

/*
s1
s2
s3
...

qry(t): how many s_i is substring of t
*/
```

5.6 SA

```
struct SA
{
    string s;
    vector <int> sa, rk, ht;
    // all 0-base including sa
    // suffix order
    // rank of suffix i
    // LCP(sf[i], sf[i+1])
    vector <int> tmp;
    int n;
    SA(string &t) {init(t);}
    inline void tSort(int m) //size of char set (base sort)
    {
        vector<int> tmp1(m + 1, 0);
        for (int i = 0; i < n; ++i) ++tmp1[rk[tmp[i]]];
        for (int i = 1; i <= m; ++i) tmp1[i] += tmp1[i - 1];
        for (int i = n - 1; i >= 0; --i) sa[--tmp1[rk[tmp[i]]]] = tmp[i];

    }
    inline void init(string &t)
    {
        s = t; n = s.size();
        sa.assign(n, 0); rk.assign(n, 0); ht.assign(n, 0); tmp.assign(n, 0);
        for (int i = 0; i < n; ++i) rk[i] = s[i], tmp[i] = i;
        int m = 127; tSort(m);
        for (int k = 1, p = 0; p < n; k <<= 1)
        {
            p = 0;
            for (int i = n - k; i < n; ++i) tmp[p++] = i;
            for (int i = 0; i < n; ++i)
                if (sa[i] >= k) tmp[p++] = sa[i] - k;
            tSort(m);
            vector <int> old_rk = rk;
            rk[sa[0]] = p = 1;
            for (int i = 1; i < n; ++i)
            {
                int a = sa[i], b = sa[i - 1];
                if (old_rk[a] == old_rk[b] && (a + k < n ? old_rk[a + k] : 0) == (b + k < n ? old_rk[b + k] : 0))
                    rk[a] = p;
                else
                    rk[a] = ++p;
            }
            m = p;
            if (p == n) break;
        }
        for (int i = 0; i < n; ++i) --rk[i];
        int h = 0;
        for (int i = 0; i < n; ++i)
        {
            if (rk[i] == 0) continue;
            int j = sa[rk[i] - 1];
            while (i + h < n && j + h < n && s[i + h] == s[j + h]) ++h;
            ht[rk[i] - 1] = h;
            if (h > 0) --h;
        }
    }
    inline void clear()
    {
        s.clear(); sa.clear(); rk.clear(); ht.clear(); tmp.clear();
        n = 0;
    }
    inline int size() const {return n;}
    inline int operator[] (int p) const {return sa[p];}
};
```

5.7 SAM

```
class SAM
{
    class node
    {
        public:int ch[26], len, fa;
        node(const int &L = 0) {memset(ch, 0, sizeof (ch)); fa = 0; len = L;}
    };
    public:
    vector <node> t; int lst;
    void GetParentTree(vector <vector <int>> &G)
    {
        G.assign(t.size(),vector<int>());
        for (unsigned i = 1; i < t.size(); ++i) G[t[i].fa].push_back(i);
    }
    void extend(const int &c)
    {
        int p = lst, np = lst = t.size();
        t.push_back(node(t[p].len + 1));
        for (; p&&!t[p].ch[c]; p = t[p].fa) t[p].ch[c] = np;
        if (!p) t[np].fa = 1;
        else
        {
            int v = t[p].ch[c];
            if (t[v].len == t[p].len + 1) t[np].fa = v;
            else
            {
                int nv = t.size(); t.push_back(t[v]);
                t[nv].len = t[p].len + 1;
                for (; p && t[p].ch[c] == v; p = t[p].fa) t[p].ch[c] = nv;
                t[np].fa = t[v].fa = nv;
            }
        }
    }
    SAM() {t.assign(2, node()); lst = 1;}
    inline void clear() {t.assign(2, node()); lst = 1;}
    inline int next(int p, int c) {return t[p].ch[c];}
    inline int Len(int p) {return t[p].len;}
};
```

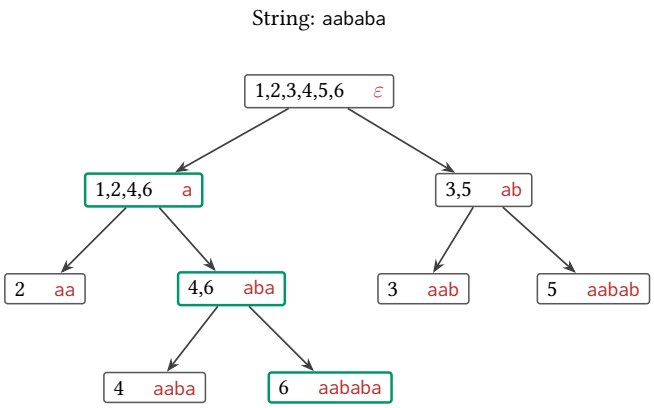


Figure 1: Suffix-link tree for the suffix automaton of aababa; green boxes mark terminal states.

6 Other

6.1 Fast I/O

```cpp
#include <bits/stdc++.h>
using namespace std;
constexpr int MAX = 1e5 + 5;
constexpr int MOD = 1e9 + 7;
constexpr int INF = 0x3f3f3f3f;
using i64 = long long;
using u64 = unsigned long long;
using pii = pair <int, int>;
using vi = vector <int>;
using vii = vector <pii>;
using LD = long double;
inline int read()
{
    int s = 0; int f = 1;
    char ch = getchar();
    while ((ch < '0' || ch > '9') && ch != EOF)
    {
        if (ch == '-') f = -1;
        ch = getchar();
    }
    while (ch >= '0' && ch <= '9')
    {
        s = s * 10 + ch - '0';
        ch = getchar();
    }
    return s * f;
}
inline void write(int x)
{
    if (x < 0) {putchar('-'); write(-x); return;}
    if (x > 9) write(x / 10);
    putchar(x % 10 + '0');
}
int main()
{
    //freopen(".in", "r", stdin);
    //freopen(".out", "w", stdout);
    return 0;
}
```

6.2 Random Number Generator

```cpp
int rd(int a, int b)
{
    static std::mt19937 gen(std::random_device{}());
    std::uniform_int_distribution <int> dist(a, b);
    return dist(gen);
}
```

6.3 Execution

```sh
#!/bin/bash
ulimit -s 102400
g++ -std=c++17 -O2 -Wall "$1" -o main
./main < in.txt > out.txt
```

```ps1
# Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
# new file : `type nul > filename.cpp`
# run : `.\run.ps1 filename.cpp`
g++ -std=c++17 -O2 -Wall $args[0] -o main
cat in.txt | .\main | Out-File -FilePath out.txt
```

6.4 Comparison

```sh
#!/bin/bash
for ((i=1; ; i++)); do
    ./gen > input.txt            # generator
    ./code < input.txt > output1.txt  # your code
    ./std < input.txt > output2.txt   # std code
    if diff output1.txt output2.txt; then  # compare
        echo "Test $i: Accepted"
    else
        echo "Test $i: Wrong Answer"
        echo "Input:"
        cat input.txt
        echo "Your Output:"
        cat output1.txt
        echo "Standard Output:"
        cat output2.txt
        break
    fi
done
```

6.5 Python

```py
import sys
sys.set_int_max_str_digits(value) # set big number
sys.setrecursionlimit(deep) # set deeper recursion
lst = sorted(lst, key = lambda x : x[1]) # sort by the second element
```